

LAPORAN TUGAS BESAR
MILESTONE III – SEMANTIC ANALYSIS
IF2224 TEORI BAHASA FORMAL DAN AUTOMATA
ARION COMPILER



Disusun oleh:

Kelompok 11

“noooRegex” - MMD

Kloce Paul William Saragih	13524040
Suryani Mulia Utami	13524042
Josh Reinhart Zidik	13524048
Syaqina Octavia Rizha	13524088

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025/2026

DAFTAR ISI

BAB 1.....	1
PENDAHULUAN.....	1
BAB 2.....	2
DASAR TEORI.....	2
2.1. Semantic Analysis.....	2
2.2. Intermediate Representation.....	2
2.3. Attributed Grammar.....	3
2.4. Abstract Syntax Tree.....	4
2.5. Decorated Abstract Syntax Tree.....	5
2.6. Symbol Table.....	5
BAB 3.....	7
PERANCANGAN.....	7
3.1. Desain Semantic Analyzer.....	7
3.2. Desain Konversi Parse Tree Menjadi AST.....	7
3.2.1. Semantic Action.....	8
3.3. Desain AST.....	13
3.4. Desain Decorated AST.....	14
3.5. Desain Symbol Table.....	14
3.5.1. Predifined Identifier.....	16
3.6. Type Compatibility Rule.....	17
3.7. Teknologi.....	19
3.8. Struktur Program.....	19
3.9. Fungsi Utama.....	20
3.10. Alur Kerja Program.....	31
3.11. Pendekatan Implementasi.....	33
BAB 4.....	35
HASIL IMPLEMENTASI.....	35
BAB 5.....	53
EKSPERIMEN.....	53
BAB 6.....	73
KESIMPULAN DAN SARAN.....	73
6.1. Kesimpulan.....	73
6.2. Saran.....	73
REFERENSI.....	74

DAFTAR GAMBAR

Gambar 1.1 Skema Kerja Interpreter.....	1
Gambar 2.3.1 Contoh Syntax Directed Translation (SDT).....	3
Gambar 2.4.1 Perbandingan Parse Tree dengan AST.....	4
Gambar 2.5.1 Contoh Decorated Abstract Syntax Tree (DAST).....	5
Gambar 3.10.1 Alur Kerja Program.....	31

DAFTAR TABEL

Tabel 3.3.1 Node-node AST terdefinisi.....	13
Tabel 3.5.1.1 Predifined Identifier.....	16
Tabel 3.6.1 Type Compability Rule.....	17
Tabel 3.2.1 Teknologi yang Digunakan.....	19
Tabel 3.9.1 Desain Fungsi Utama.....	20
Tabel 5.1 Kasus Uji 1.....	53
Tabel 5.2 Kasus Uji 2.....	55
Tabel 5.3 Kasus Uji 3.....	58
Tabel 5.4 Kasus Uji 4.....	59
Tabel 5.5 Kasus Uji 5.....	59
Tabel 5.6 Kasus Uji 6.....	60
Tabel 5.7 Kasus Uji 7.....	64
Tabel 5.8 Kasus Uji 8.....	67

BAB 1

PENDAHULUAN

Bahasa pemrograman digunakan untuk menuliskan instruksi agar komputer dapat menjalankan suatu program tanpa harus menggunakan bahasa mesin secara langsung. Komputer hanya memahami bahasa mesin dalam bentuk biner (0 dan 1), sehingga bahasa pemrograman dikembangkan agar lebih mudah dipahami manusia. Secara umum, bahasa pemrograman dibedakan menjadi low-level yang dekat dengan bahasa mesin dan high-level yang lebih mendekati bahasa manusia. Semakin tinggi tingkat bahasanya, semakin banyak proses yang diperlukan untuk mengubahnya menjadi kode mesin.

Salah satu cara untuk menerjemahkan bahasa pemrograman ke dalam kode mesin adalah dengan menggunakan interpreter. Interpreter bekerja dengan membaca, menerjemahkan, dan mengeksekusi kode sumber secara baris demi baris. Pendekatan ini memudahkan dalam menemukan kesalahan karena error dapat langsung diketahui pada baris tertentu. Berbeda dengan compiler yang menerjemahkan seluruh source code menjadi kode mesin (misalnya dalam bentuk object file) sebelum program dijalankan.

Meskipun berbeda dalam cara eksekusi, keduanya tetap melalui tahapan analisis yang serupa. Proses dimulai dari Lexical Analysis yang memecah kode sumber menjadi token-token kecil seperti kata kunci, variabel, dan operator. Selanjutnya, Syntax Analysis yang memeriksa apakah susunan token tersebut sudah sesuai dengan aturan tata bahasa (*grammar*) yang berlaku. Setelah itu, Semantic Analysis memastikan bahwa makna dari kode tersebut benar. Tahap terakhir, Intermediate Code Generation yang mengubah kode ke bentuk perantara yang lebih mudah diproses sebelum akhirnya dieksekusi atau diterjemahkan lebih lanjut menjadi kode mesin.



Gambar 1.1 Skema Kerja Interpreter

Sumber: GeeksForGeeks

BAB 2

DASAR TEORI

2.1. Semantic Analysis

Semantic Analysis adalah proses pengecekan makna (*semantic*) dari program untuk memastikan program tidak hanya benar secara sintaksis, tetapi juga benar secara logika dan aturan bahasa pemrograman. Setelah parser memastikan susunan token mengikuti grammar dan menghasilkan parse tree, Semantic Analysis akan memverifikasi apakah program tersebut masuk akal untuk dijalankan. Pengecekan yang dilakukan antara lain: (1) apakah variabel sudah dideklarasikan sebelum digunakan, (2) apakah tipe data pada operasi kompatibel, (3) apakah akses variabel sesuai scope, serta (4) apakah struktur kontrol program valid.

Dalam prosesnya, Semantic Analysis terdiri atas beberapa tahap di antaranya:

1. **Type Checking** (Pengecekan Tipe): Memastikan operator dan operan memiliki tipe yang kompatibel.
2. **Symbol Table Management** (Pengecekan Deklarasi) : Mengelola deklarasi identifier seperti variabel, konstanta, prosedur, dan fungsi.
3. **Scope Resolution** (Pengecekan Lingkup): Menentukan validitas akses identifier berdasarkan scope program.
4. **Control Flow Validation** (Pengecekan Alur Kontrol): Memastikan struktur alur program valid secara logika.

Semantic Analysis bekerja di atas parse tree yang dihasilkan Parser dan menghasilkan struktur baru berupa Decorated Abstract Syntax Tree (**Decorated AST**), yaitu AST yang telah dianotasi dengan informasi semantik.

2.2. Intermediate Representation

Intermediate Representation (IR) adalah representasi program yang berada di antara source code dan target language. Dengan adanya IR, compiler dapat melakukan translasi source code ke target code, dengan terlebih dahulu membangun model konseptual program yang lebih mudah dianalisis dan diproses lebih lanjut.

IR dapat direpresentasikan dalam berbagai bentuk, di antaranya tree dan graph. Salah satu bentuk IR yang paling umum digunakan adalah Abstract Syntax Tree (AST). AST berbentuk tree, namun setelah melalui proses Semantic Analysis dan dianotasi menjadi Decorated AST, identifier seperti variabel, fungsi, dan tipe data dapat direferensikan oleh banyak node

sekaligus melalui symbol table. Karena satu entitas dapat memiliki banyak referensi dari berbagai node, secara konseptual struktur tersebut tidak lagi murni berbentuk tree, melainkan graph.

Untuk memproses IR, traversal node dilakukan menggunakan mekanisme visitor dengan pendekatan seperti *preorder*, *inorder*, atau *postorder*, sesuai kebutuhan analisis maupun evaluasi program.

2.3. Attributed Grammar

Attributed Grammar adalah perluasan dari Context-Free Grammar (CFG) yang melengkapi setiap simbol dalam grammar dengan:

1. **Atribut**, yaitu nilai-nilai yang membawa informasi semantik. Setiap simbol dalam produksi grammar dapat memiliki atribut seperti tipe data, nilai, atau scope.
2. **Aturan semantik** (*Semantic action* / *Translation rule*), yang mendefinisikan bagaimana atribut tersebut dihitung. Aturan semantik menentukan bagaimana atribut suatu node diturunkan dari node anaknya (*synthesized attributes*) atau diwariskan dari node induknya (*inherited attributes*).

Syntax-Directed Translation (SDT) merupakan salah satu bentuk penerapan Attributed Grammar yang berfokus pada proses translasi untuk menghasilkan representasi baru dari program. Pada SDT, setiap produksi grammar dilengkapi dengan *semantic action* yang mendefinisikan bagaimana hasil translasi suatu nonterminal dihitung berdasarkan translasi simbol-simbol di sisi kanan produksi. Translasi dilakukan secara *bottom-up* pada parse tree karena translasi suatu node dapat bergantung pada hasil translasi *child node*-nya. Dalam implementasi compiler ini, hasil translasi tersebut berupa pembentukan node-node **Abstract Syntax Tree (AST)** yang kemudian dirakit menjadi struktur AST yang utuh.

CFG Production	Translation rules
$exp \rightarrow exp \text{ PLUS } term$	$exp_1.trans = \text{new PlusNode}(exp_2, term.trans)$
$exp \rightarrow term$	$exp.trans = term.trans$
$term \rightarrow term \text{ TIMES } factor$	$term_1.trans = \text{new TimesNode}(term_2.trans, factor.trans)$
$term \rightarrow factor$	$term.trans = factor.trans$
$factor \rightarrow \text{INTLITERAL}$	$factor.trans = \text{new IntLitNode}(\text{INTLITERAL.value})$
$factor \rightarrow \text{LPARENS } exp \text{ RPARENS}$	$factor.trans = exp.trans$

Gambar 2.3.1 Contoh Syntax Directed Translation (SDT)

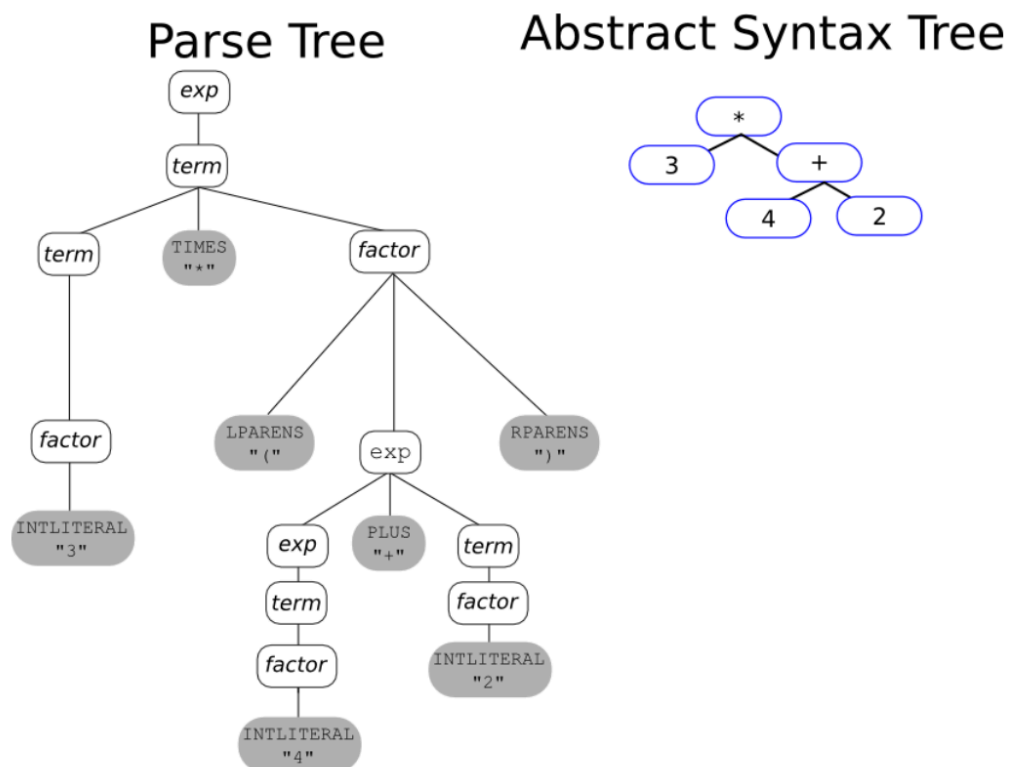
Sumber: <https://pages.cs.wisc.edu/~hasti/cs536/readings/SDT.html>

Untuk transition rules yang pertama, berarti ketika ditemukan pola *exp PLUS term*, bangun sebuah *PlusNode* yang anak kirinya adalah atribut

.trans dari exp anak, dan anak kanannya adalah atribut .trans dari term. Hasilnya disimpan sebagai atribut .trans dari exp induk. Atribut .trans di sini adalah synthesized attribute — nilainya dihitung dari bawah ke atas

2.4. Abstract Syntax Tree

Abstract Syntax Tree (AST) adalah bentuk IR yang merepresentasikan struktur semantik suatu program dalam bentuk pohon. Berbeda dengan parse tree yang mencerminkan struktur sintaksis grammar secara literal, AST hanya mempertahankan node-node yang memiliki makna semantik, seperti operasi, nilai, deklarasi, dan pernyataan. Elemen-elemen sintaksis murni yang tidak membawa makna dan node-node perantara grammar, dihilangkan.



Gambar 2.4.1 Perbandingan Parse Tree dengan AST

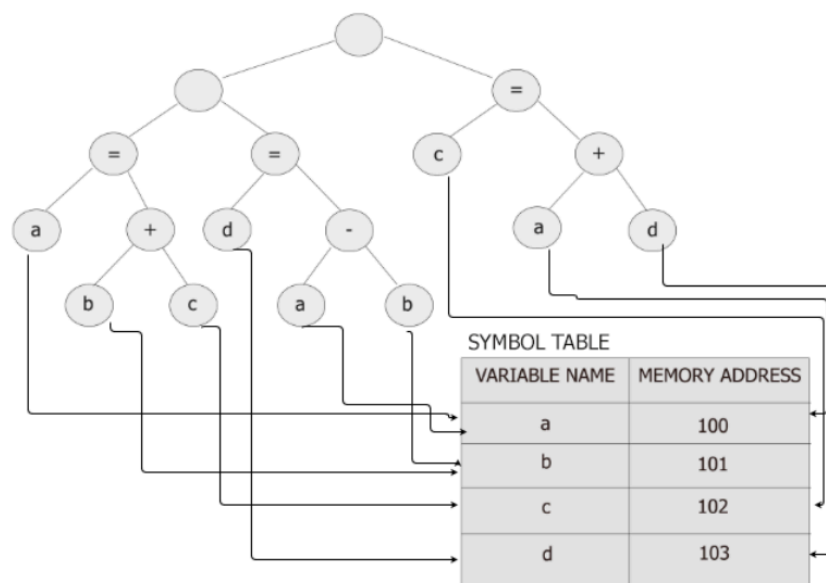
Sumber: <https://pages.cs.wisc.edu/~hasti/cs536/readings/SDT.html>

Dengan struktur yang lebih ringkas ini, AST menjadi representasi yang lebih efisien dan lebih mudah diproses oleh tahap-tahap berikutnya. Setelah AST terbentuk, informasi semantik seperti tipe data, scope, dan lexical level setiap identifier perlu disimpan secara terpusat agar dapat diakses berulang kali selama analisis pada **Symbol Table**. Proses pengisian Symbol Table dan penambahan anotasi semantik pada node-node AST dilakukan melalui

penelusuran pohon menggunakan fungsi visit, dan hasil akhirnya adalah **Decorated AST**.

2.5. Decorated Abstract Syntax Tree

Decorated Abstract Syntax Tree (DAST) adalah AST yang telah diberi anotasi semantik sebagai hasil dari proses Semantic Analysis. Anotasi tersebut dapat berupa informasi tipe data, referensi identifier ke symbol table, lexical level, dan scope. Karena Decorated AST memuat sekaligus struktur program dan informasi semantiknya, ia menjadi IR yang paling siap digunakan oleh tahap berikutnya..



Gambar 2.5.1 Contoh Decorated Abstract Syntax Tree (DAST)

Sumber: <https://silcnitc.github.io/data-structures.html>

2.6. Symbol Table

Symbol Table adalah struktur data yang digunakan oleh Semantic Analyzer untuk menyimpan dan mengelola informasi mengenai setiap identifier yang dideklarasikan dalam program. Informasi yang disimpan mencakup nama identifier, jenis objek (kind), tipe data, scope, dan lexical level. Dengan adanya Symbol Table, Semantic Analyzer dapat memverifikasi apakah suatu identifier sudah dideklarasikan sebelum digunakan, serta memastikan bahwa penggunaannya sesuai dengan deklarasinya.

Karena scope dalam bahasa pemrograman bersifat *nested*, di mana scope yang lebih dalam dapat mengakses identifier dari scope yang lebih luar, namun tidak sebaliknya, Symbol Table diimplementasikan menggunakan

struktur data *stack*. Setiap kali program memasuki blok baru seperti prosedur atau fungsi, entri scope baru didorong (*push*) ke *stack*. Saat keluar dari blok tersebut, entri scope tersebut dikeluarkan (*pop*) sehingga identifier yang dideklarasikan di dalamnya tidak lagi dapat diakses. Operasi pencarian (*lookup*) dilakukan dari scope terdalam hingga terluar, sehingga deklarasi di scope yang lebih dalam akan diprioritaskan apabila terdapat identifier dengan nama yang sama di scope yang berbeda.

Selain identifier yang dideklarasikan oleh programmer, Symbol Table juga memuat predefined identifier, yaitu identifier yang sudah terdefinisi sebelum program mulai dieksekusi. Predefined identifier mencakup tipe data primitif bawaan bahasa seperti integer, real, boolean, dan char, serta prosedur dan fungsi standar yang dapat dipanggil tanpa deklarasi eksplisit. Entri-entri ini diinisialisasi ke dalam Symbol Table sebelum proses analisis dimulai, sehingga Semantic Analyzer dapat mengenali dan memvalidasi penggunaannya di dalam program.

BAB 3

PERANCANGAN

3.1. Desain Semantic Analyzer

Masukan	Parse tree
Keluaran	Decorated Abstract Syntax Tree (DAST) beserta Symbol Table yang telah terisi informasi semantik program.
Algoritma	L-Attributed Grammar , di mana pencatatan atribut dihitung dalam satu kali traversal <i>left-to-right, top-down</i>

Secara umum, Semantic Analyzer bekerja dalam dua tahap utama:

1. **Konversi Parse Tree** menjadi AST menggunakan pendekatan Syntax-Directed Translation (SDT).
2. **Penelusuran AST** untuk melakukan pengecekan semantic serta anotasi node-node AST menjadi Decorated AST.

Proses analisis dilakukan menggunakan traversal berbasis *visitor* dengan pendekatan Depth-First Search (DFS). Selama traversal, Semantic Analyzer akan:

- ◆ melakukan lookup identifier pada Symbol Table,
- ◆ memvalidasi scope identifier,
- ◆ melakukan inferensi dan pengecekan tipe data,
- ◆ mengelola lexical scope,
- ◆ serta menambahkan anotasi semantik pada node AST.

3.2. Desain Konversi Parse Tree Menjadi AST

Digunakan *Syntax-Directed Translation* (SDT) yang mana untuk setiap aturan produksi grammar, di-*augment* dengan *semantic action*. Aturan tersebut kemudian dieksekusi pada setiap node-node Parse tree secara *bottom-up* untuk membangun node-node AST. Selain itu, akan dihilangkan juga informasi sintaksis yang sudah tidak diperlukan untuk tahap analisis. Terakhir, menggabungkan potongan node dari bawah ke atas dengan node level bawah akan diserahkan ke node induknya untuk dirakit menjadi satu kesatuan AST yang utuh.

3.2.1. Semantic Action

1. `<program> -> program-header + declaration-part + compound-statement + period`
`program.trans = new ProgramNode(header.name, decl.trans, compd.trans)`
2. `<program-header> -> programsy + ident + semicolon`
`header.name = ident.value`
3. `<declaration-part> -> (const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*`
`decl.trans = flatten([constDecl.trans*, typeDecl.trans*, varDecl.trans*, subprog.trans*])`
4. `<const-declaration> -> constsy + (ident + eql + constant + semicolon)+`
`for each (ident, constant): constDecl.trans += new ConstDeclNode(ident.value, constant.trans)`
5.
 - a. `<constant> -> charcon`
`constant.trans = new LiteralNode(Char, charcon.value)`
 - b. `<constant> -> string`
`constant.trans = new LiteralNode(String, string.value)`
 - c. `<constant> -> (plus|minus)? (ident|intcon|realcon)`
`constant.trans = new LiteralNode(kind, value) (dengan UnaryOpNode jika ada tanda)`
6. `<type-declaration> -> typesy + (ident + eql + type + semicolon)+`
`for each (ident, type): typeDecl.trans += new TypeDeclNode(ident.value, type.trans)`
7. `<var-declaration> -> varsy + (identifier-list + colon + type + semicolon)+`
`for each (identifier-list, type): varDecl.trans += new VarDeclNode(identList.trans, type.trans)`
8. `<identifier-list> -> ident (comma + ident)*`
`identList.trans = [ident.value*]`
9.
 - a. `<type> -> ident`
`type.trans = new SimpleTypeNode(ident.value)`
 - b. `<type> -> array-type`
`type.trans = arrayType.trans`
 - c. `<type> -> range`

```

        type.trans = range.trans

d. <type> -> enumerated

        type.trans = enumerated.trans

e. <type> -> record-type

        type.trans = recordType.trans

10. <array-type> -> arraysy + lbrack + (range | ident) + rbrack + ofsy + type

        arrayType.trans = new ArrayTypeNode(type.trans, index.trans)

11. <range> -> constant + period + period + constant

        range.trans = new RangeTypeNode(constant1.trans, constant2.trans)

12. <enumerated> -> lparent + ident + (comma + ident)* + rparent

        enumerated.trans = new EnumTypeNode([ident.value*])

13. <record-type> -> recordsy + field-list + endsy

        recordType.trans = new FieldTypeNode(fieldList.trans)

14. <field-list> -> field-part + (semicolon + field-part)*

        fieldList.trans = [fieldPart.trans*]

15. <field-part> -> identifier-list + colon + type

        fieldPart.trans = (identList.trans, type.trans)

16. <subprogram-declaration> -> procedure-declaration | function-declaration

        subprog.trans = child.trans

17. <procedure-declaration> -> proceduresy + ident + (formal-parameter-list)? +
    semicolon + block + semicolon

        procDecl.trans = new ProcDeclNode(ident.value, formalParams.trans,
        block.decls, block.body)

18. <function-declaration> -> functionsy + ident + (formal-parameter-list)? +
    colon + ident + semicolon + block + semicolon

        funcDecl.trans = new FuncDeclNode(ident.value, returnType.trans,
        formalParams.trans, block.decls, block.body)

19. <block> -> declaration-part + compound-statement

        block.decls = decl.trans; block.body = compound.trans

20. <formal-parameter-list> -> lparent + parameter-group + (semicolon +
    parameter-group)* + rparent

        formalParams.trans = [paramGroup.trans*]

21. <parameter-group> -> identifier-list + colon + (ident | array-type)

```

```

    paramGroup.trans = new ParamDeclNode(identList.trans, type.trans)
22. <compound-statement> -> beginsy + statement-list + endsy
    compound.trans = new CompoundNode(stmtList.trans)
23. <statement-list> -> statement (semicolon + statement)*
    stmtList.trans = [stmt.trans*]
24.
    a. <statement> -> assignment-statement
        stmt.trans = assignStmt.trans
    b. <statement> -> if-statement
        stmt.trans = ifStmt.trans
    c. <statement> -> case-statement
        stmt.trans = caseStmt.trans
    d. <statement> -> while-statement
        stmt.trans = whileStmt.trans
    e. <statement> -> repeat-statement
        stmt.trans = repeatStmt.trans
    f. <statement> -> for-statement
        stmt.trans = forStmt.trans
    g. <statement> -> procedure/function-call
        stmt.trans = new CallStmtNode(call.trans)
    h. <statement> -> ε
        stmt.trans = new CompoundNode([])
25.
    a. <variable> -> ident
        variable.trans = new VarRefNode(ident.value)
    b. <variable> -> ident (component-variable)+ (last = [index])
        variable.trans = new ArrayAccessNode(base.trans, index.trans)
        (left-recursive)
    c. <variable> -> ident (component-variable)+ (last = .field)
        variable.trans = new FieldAccessNode(base.trans, ident.value)
26.
    a. <component-variable> -> (lbrack + index-list + rbrack)

```

```

        compVar.trans = new ArrayAccessNode(current.trans, index.trans)
        (dichain kiri ke kanan)

    b. <component-variable> -> (period + ident)

        compVar.trans = new FieldAccessNode(current.trans, ident.value)
        (dichain kiri ke kanan)

27. <index-list> -> ( intcon | charcon | ident ) + (comma + index-list)*

    indexList.trans = [expr.trans*]

28. <assignment-statement> -> variable + becomes + expression

    assign.trans = new AssignNode(variable.trans, expr.trans)

29. <if-statement> -> ifsy + expression + thensty + statement + (elsy +
    statement)?

    if.trans = new IfNode(expr.trans, thenStmt.trans, elseStmt.trans?)

30. <case-statement> -> casesy + expression + ofsy + case-block + endsy

    case.trans = new CaseNode(expr.trans, caseBlock.trans)

31. <case-block> -> constant + (comma + constant)* + colon + statement +
    (semicolon + case-block?)*

    for each block: caseBlock.trans += ([constant.trans*], stmt.trans)

32. <while-statement> -> whilesy + expression + dosy + compound-statement +
    semicolon

    while.trans = new WhileNode(expr.trans, compound.trans)

33. <repeat-statement> -> repeatsy statement-list untilsy expression

    repeat.trans = new RepeatNode(stmtList.trans, expr.trans)

34. <for-statement> -> forsy ident becomes expression (tosy|downtosy) expression
    dosy compound-statement semicolon

    for.trans = new ForNode(goesUp, ident.value, startExpr.trans, endExpr.trans,
    compound.trans)

35. <procedure/function-call> -> ident lparent parameter-list? rparent

    call.trans = new CallNode(ident.value, paramList.trans)

36. <parameter-list> -> expression (comma expression)*

    paramList.trans = [expr.trans*]

37.
    a. <expression> -> simple-expression relational-operator
        simple-expression

        expr.trans = new BinaryOpNode(relOp.value, simpleExpr1.trans,
        simpleExpr2.trans)

    b. <expression> -> simple-expression

```

```
expr.trans = simpleExpr.trans
```

38.

a. <simple-expression> -> (plus|minus) term (additive-operator term)*

```
simpleExpr.trans = new UnaryOpNode(sign.value, term.trans) (lalu:  
untuk setiap additive-op: simpleExpr.trans = new  
BinaryOpNode(op.value, simpleExpr.trans, term.trans))
```

b. <simple-expression> -> term (additive-operator term)*

```
simpleExpr.trans = term.trans (lalu: untuk setiap additive-op:  
simpleExpr.trans = new BinaryOpNode(op.value, simpleExpr.trans,  
term.trans))
```

39. <term> -> factor (multiplicative-operator factor)*

```
term.trans = factor.trans (lalu: untuk setiap mul-op: term.trans = new  
BinaryOpNode(op.value, term.trans, factor.trans))
```

40.

a. <factor> -> intcon

```
factor.trans = new LiteralNode(Int, intcon.value)
```

b. <factor> -> realcon

```
factor.trans = new LiteralNode(Real, realcon.value)
```

c. <factor> -> charcon

```
factor.trans = new LiteralNode(Char, charcon.value)
```

d. <factor> -> string

```
factor.trans = new LiteralNode(String, string.value)
```

e. <factor> -> notsy factor

```
factor.trans = new UnaryOpNode("not", factor.trans)
```

f. <factor> -> lparent expression rparent

```
factor.trans = expression.trans
```

g. <factor> -> procedure/function-call

```
factor.trans = call.trans
```

h. <factor> -> variable

```
factor.trans = variable.trans
```

41. <relational-operator> -> eql|neq|gtr|geq|lss|leq

```
relOp.value = token.value
```

42. <additive-operator> -> plus|minus|orsy

```
addOp.value = token.value
```

```

43. <multiplicative-operator> -> times|rdiv|idiv|imod|andsy

mulOp.value = token.value

```

3.3. Desain AST

AST terdiri atas beberapa jenis node utama yang dikelompokkan berdasarkan kebutuhan representasi program. Setiap node AST menyimpan child node yang merepresentasikan hubungan antar operasi, operan, deklarasi, maupun statement.

Secara lengkap, node-node AST yang didefinisikan pada program adalah sebagai berikut.

Tabel 3.3.1 Node-node AST terdefinisi

Node AST	Jenis
SimpleTypeNode ArrayTypeNode EnumTypeNode RangeTypeNode FieldTypeNode	Types
LiteralNode CallNode BinaryOpNode UnaryOpNode VarRefNode ArrayAccessNode FieldAccessNode	Expressions
ConstDeclNode TypeDeclNode VarDeclNode ParamDeclNode ProcDeclNode FuncDeclNode	Declarations
AssignNode	Statements

IfNode WhileNode ForNode RepeatNode CaseNode CompoundNode CallStmtNode	
ProgramNode	Root

3.4. Desain Decorated AST

Untuk membangun Decorated AST, ditambahkan anotasi semantik selama traversal AST menggunakan semantic visitor. Informasi semantik yang ditambahkan pada node Decorated AST antara lain:

- tipe data node, disimpan via type cache dan ditampilkan pada setiap node ekspresi,
- lexical level, yang merepresentasikan kedalaman scope tempat identifier dideklarasikan,
- referensi ke Symbol Table berupa indeks yang mengarah ke entri identifier pada tabel tab.

Selama traversal juga akan dilakukan pengecekan kompatibilitas tipe, apabila ditemukan pelanggaran, error semantik dicatat dan dilaporkan tanpa menghentikan proses traversal secara keseluruhan.

3.5. Desain Symbol Table

Symbol Table digunakan untuk menyimpan informasi semantik mengenai identifier yang muncul pada program. Dalam program ini, Symbol Table dibagi menjadi tiga tabel utama, yaitu tab, atab, dan btab. Selain itu, digunakan juga struktur display dan variabel level untuk mengelola scope aktif selama proses semantic analysis.

➤ **tab**

Tabel **tab** digunakan untuk menyimpan informasi identifier seperti reserved word, konstanta, variabel, tipe, fungsi, dan prosedur. Setiap entri pada **tab** memiliki atribut:

- identifier (id),
- jenis objek (obj),
- tipe data (type),
- lexical level (lev),
- alamat atau offset memori (adr),
- link identifier dalam scope tertentu (link),
- nilai konstanta (const_value) untuk konstanta,
- serta referensi (ref) ke tabel lain apabila identifier memiliki tipe kompleks.

Tabel **tab** menjadi pusat utama *lookup* semantic information selama proses semantic analysis.

➤ **atab**

Tabel **atab** digunakan untuk menyimpan informasi tipe array. Setiap entri pada **atab** menyimpan tipe index (xtyp), tipe elemen (etyp), referensi elemen kompleks (eref), batas bawah (low) dan batas atas (high) index, ukuran elemen (elsz), serta total ukuran array (size).

Jika suatu identifier bertipe array, maka field **ref** pada **tab** akan mengarah ke entri array yang bersesuaian pada **atab**.

➤ **btab**

Tabel **btab** digunakan untuk menyimpan informasi block program seperti procedure, function, record, atau scope tertentu. Setiap entri pada **btab** menyimpan identifier terakhir dalam block (last), parameter terakhir (lpar), ukuran parameter (psze), serta ukuran variabel lokal (vsze) pada block tersebut.

Jika suatu identifier merepresentasikan procedure, function, record, atau block scope tertentu, maka field **ref** pada **tab** akan mengarah ke entri block yang bersesuaian pada **btab**.

➤ **Manajemen scope**

Manajemen scope dilakukan menggunakan level dan display. Ketika program memasuki scope baru, fungsi **enterScope()** akan menambahkan block baru ke **btab**, menaikkan level, dan memperbarui display. Ketika keluar dari scope, fungsi **exitScope()** akan menurunkan level dan mengembalikan scope aktif ke scope sebelumnya. Dengan struktur ini, pencarian identifier dapat dilakukan dari scope terdalam

menuju scope terluar menggunakan `searchTab()`, sedangkan pengecekan deklarasi ulang dalam scope yang sama dilakukan menggunakan `searchCurrentScope()`.

Dalam implementasi ini, node-node pada Decorated AST mereferensikan entri pada `tab`. Selanjutnya, field `ref` pada `tab` digunakan untuk mengakses informasi tambahan pada `atab` atau `btab` apabila identifier memiliki tipe kompleks atau merepresentasikan block tertentu.

3.5.1. Predifined Identifier

Tabel 3.5.1.1 Predifined Identifier

Identifier	Obj	Type	Keterangan
real	OBJ_TYPE	TYPE_REAL	Memiliki type Real e.g. someArray[1..10] of Real
integer	OBJ_TYPE	TYPE_INTEGER	Memiliki type Integer e.g. procedure print(x: integer);
char	OBJ_TYPE	TYPE_CHAR	Memiliki type Char e.g. var a: Char
boolean	OBJ_TYPE	TYPE_BOOLEAN	Memiliki type Boolean e.g. var b: Boolean
string	OBJ_TYPE	TYPE_STRING	Memiliki type String e.g. var c: String
array	OBJ_TYPE	TYPE_ARRAY	Memiliki type Array. e.g. var x: array [1..10] of integer

record	OBJ_TYPE	TYPE_RECORD	Memiliki type Record. e.g. type Point = record x: integer; y: integer end
true	OBJ_CONST	TYPE_BOOLEAN	Memiliki type Boolean dengan nilai true e.g. const isTrue == true
false	OBJ_CONST	TYPE_BOOLEAN	Memiliki type Boolean dengan nilai false e.g. if (someVar = false) then
readln	OBJ_PROC	TYPE_NONE	Prosedur standar untuk membaca input. e.g. readln(x)
writeln	OBJ_PROC	TYPE_NONE	Prosedur standar untuk mencetak output. e.g. writeln(x)

3.6. Type Compatibility Rule

Tabel 3.6.1 Type Compability Rule

Nama Type	Keterangan
Simple type	
Real	realcon memiliki type Real
Integer	intcon memiliki type Integer

Char	charcon memiliki type Char
Boolean	true dan false memiliki type Boolean. Term dan expression yang diproduksi dari notsy, andsy, dan orsy memiliki type Boolean. Expression yang diproduksi dari relational-operator (eq1, neq, gtr, geq, lss, leq) memiliki type Boolean
String	String string memiliki type String
Structured type	
Range	Type dari Node Range ditentukan oleh type dari Constant di dalamnya. Lower Bound dari Range (Constant di kiri) tidak boleh lebih besar daripada Upper Bound dari Range. Subrange tidak boleh memiliki type Real.
Enumerated	Type dari Node Enumerated ditentukan oleh type dari Ident di dalamnya. Ident di dalam enumerated harus memiliki type yang sama.
Array	Array memiliki struktur berikut: array [T1] of T2 Dengan T1 adalah index type dan T2 adalah element type. Index type harus memiliki simple type bukan Real. Sedangkan T2 dapat berupa type apapun.
Record	Field di dalam Record dapat memiliki type apapun

3.7. Teknologi

Berikut adalah teknologi yang digunakan dalam program:

Tabel 3.2.1 Teknologi yang Digunakan

Teknologi yang digunakan	Penjelasan
Bahasa Pemrograman C++ dan STL Library	Digunakan untuk mengimplementasikan lexer, DFA, parser, ASTer, semantic analyzer, dan seluruh logika program. Program ini menggunakan library standar C++ seperti fstream untuk pengolahan file; map, set, dan vector untuk pengelolaan data, memory untuk manajemen node parse tree dan AST secara otomatis, serta unordered_map untuk type cache pada proses anotasi Decorated AST.
Build Tool Makefile	Digunakan untuk mengatur proses kompilasi program secara otomatis.

3.8. Struktur Program

Berikut adalah struktur dari program:

MMD-TUBES-IF2224-2026/

```
|
|— doc/
|— src/
|   |— main.cpp
|   |— Symbol.cpp
|   |— Symbol.hpp
|   |— Token.cpp
|   |— Token.hpp
|   |
|   |— Lexer/
|       |— dfa_rules.txt
|       |— DFA.cpp
|       |— DFA.hpp
|       |— lexer.cpp
|       |— lexer.hpp
```

```

|
|
|— Parser/
|   |— Parser.cpp
|   |— Parser.hpp
|   |— ParseTree.cpp
|   |— ParseTree.hpp
|
|— Semantic/
|   |— AST.cpp
|   |— AST.hpp
|   |— ASTer.cpp
|   |— ASTer.hpp
|   |— SemanticAnalyzer.cpp
|   |— SemanticAnalyzer.hpp
|   |— SymbolTable.cpp
|   |— SymbolTable.hpp
|
|— test/
|   |— milestone-1/
|   |— milestone-2/
|   |— milestone-3/
|
|— README.md
|— Makefile

```

3.9. Fungsi Utama

Tabel 3.9.1 Desain Fungsi Utama

Nama Kelas	Data/Method	Penjelasan
ASTer	buildAST(root)	Method utama yang memulai proses pembangunan AST dengan memanggil buildProgramNode(root). Mengembalikan shared_ptr<ASTNode> yang merupakan root dari AST.
	buildTypeNode(root)	Mendispatch pembangunan TypeNode berdasarkan tipe anak pertama dari node TYPE. Mendelegasikan ke buildSimpleType, buildArrayType,

		buildRangeType, buildEnumType, atau buildRecordType.
	buildSimpleType(node)	Membangun SimpleTypeNode dari sebuah node ident. Digunakan untuk tipe dasar seperti integer, real, boolean, char, string, atau nama tipe yang didefinisikan pengguna.
	buildArrayType(node)	Membangun ArrayTypeNode dari node ARRAY_TYPE. Mengekstrak tipe indeks (range atau simple) dan tipe elemen, lalu membentuk ArrayTypeNode.
	buildRangeType(node)	Membangun RangeTypeNode dari node RANGE. Mengekstrak dua node CONSTANT untuk batas bawah dan atas, lalu mengonversinya menjadi LiteralNode.
	buildEnumType(node)	Membangun EnumTypeNode dari node ENUMERATED dengan mengumpulkan semua ident anak menjadi daftar string nilai enumerasi.
	buildRecordType(node)	Membangun FieldTypeNode dari node RECORD_TYPE. Mengiterasi FIELD_PART dalam field-list dan mengonversi masing-masing menjadi pasangan (nama-nama, TypeNode).
	buildDeclNodes(root)	Mendispach pembangunan DeclNode dari node DECLARATION_PART. Mengenali anak CONST_DECLARATION, TYPE_DECLARATION, VAR_DECLARATION, dan SUBPROGRAM_DECLARATION, lalu mengumpulkan hasilnya.
	buildConstDeclNode(root)	Membangun vektor ConstDeclNode dari CONST_DECLARATION. Setiap entri dihasilkan dari triplet ident + eql + constant dengan langkah 4.
	buildTypeDeclNode(root)	Membangun vektor TypeDeclNode dari TYPE_DECLARATION. Setiap entri terdiri dari nama ident dan TypeNode yang dibangun dari node TYPE.

buildVarDeclNode(root)	Membangun vektor VarDeclNode dari VAR_DECLARATION. Setiap entri berisi daftar nama dari identifier-list dan TypeNode dari node TYPE.
buildParamDeclNode(root)	Membangun ParamDeclNode tunggal dari PARAMETER_GROUP. Mengekstrak daftar nama dan tipe; is_var selalu false pada implementasi ini.
buildFormalParams(root)	Mengumpulkan semua PARAMETER_GROUP dari FORMAL_PARAMETER_LIST menjadi vektor ParamDeclNode.
buildProcDeclNode(root)	Membangun ProcDeclNode dari PROCEDURE_DECLARATION. Mengekstrak nama, parameter formal, deklarasi lokal, dan body (CompoundNode).
buildFuncDeclNode(root)	Membangun FuncDeclNode dari FUNCTION_DECLARATION. Selain nama, parameter, dan body, juga mengekstrak tipe kembalian sebagai SimpleTypeNode.
buildLiteralNode(root)	Membangun LiteralNode dari terminal intcon, realcon, charcon, atau string.
buildCallNode(root)	Membangun CallNode dari PROCEDURE_FUNCTION_CALL. Mengekstrak nama fungsi/prosedur dan semua argumen (EXPRESSION atau PARAMETER_LIST) sebagai vektor ExprNode.
buildBinaryOpNode(lhs, op, rhs)	Membangun BinaryOpNode dari dua ExprNode dan sebuah node operator. Nama operator diekstrak dari anak pertama node operator menggunakan toString(Symbol).
buildUnaryOpNode(op, operand)	Membangun UnaryOpNode dari node operator (misalnya notsy, minus) dan operand ExprNode.
buildVarRefNode(root)	Membangun VarRefNode dari node VARIABLE yang hanya memiliki satu anak ident (tanpa komponen).

	<code>buildArrayAccessNode(root)</code>	Membangun <code>ArrayAccessNode</code> dari <code>VARIABLE</code> dengan komponen <code>lbrack</code> . Mendukung akses multi-dimensi dengan melakukan wrapping bertingkat.
	<code>buildFieldAccessNode(root)</code>	Membangun <code>FieldAccessNode</code> dari <code>VARIABLE</code> dengan komponen <code>period</code> . Mendukung akses field bertingkat pada record.
	<code>buildExprNode(root)</code>	Dispatcher utama untuk ekspresi. Mendispatch ke builder yang tepat berdasarkan tipe node <code>EXPRESSION</code> , <code>SIMPLE_EXPRESSION</code> , <code>TERM</code> , atau <code>FACTOR</code> .
	<code>buildStmtNode(root)</code>	Dispatcher utama untuk pernyataan. Mendispatch ke <code>buildAssignNode</code> , <code>buildIfNode</code> , <code>buildWhileNode</code> , <code>buildForNode</code> , <code>buildRepeatNode</code> , <code>buildCaseNode</code> , <code>buildCompoundNode</code> , atau <code>CallNode</code> .
	<code>buildAssignNode(root)</code>	Membangun <code>AssignNode</code> dari <code>ASSIGNMENT_STATEMENT</code> . Mengekstrak sisi kiri (<code>VARIABLE</code>) dan sisi kanan (<code>EXPRESSION</code>).
	<code>buildIfNode(root)</code>	Membangun <code>IfNode</code> dari <code>IF_STATEMENT</code> . Mengekstrak kondisi, blok <code>then</code> , dan (opsional) blok <code>else</code> .
	<code>buildWhileNode(root)</code>	Membangun <code>WhileNode</code> dari <code>WHILE_STATEMENT</code> . Mengekstrak kondisi dan body (<code>COMPOUND_STATEMENT</code>).
	<code>buildForNode(root)</code>	Membangun <code>ForNode</code> dari <code>FOR_STATEMENT</code> . Mengekstrak variabel kontrol, nilai awal, nilai akhir, arah (<code>to/downto</code>), dan body.
	<code>buildRepeatNode(root)</code>	Membangun <code>RepeatNode</code> dari <code>REPEAT_STATEMENT</code> . Mengekstrak body (<code>STATEMENT_LIST</code>) dan kondisi <code>until</code> (<code>EXPRESSION</code>).

	<code>buildCaseNode(root)</code>	Membangun <code>CaseNode</code> dari <code>CASE_STATEMENT</code> . Mengekstrak kunci ekspresi dan pasangan (label-list, statement) dari setiap <code>CASE_BLOCK</code> .
	<code>buildCompoundNode(root)</code>	Membangun <code>CompoundNode</code> dari <code>COMPOUND_STATEMENT</code> dengan mendelegasikan isi <code>STATEMENT_LIST</code> ke <code>buildStatementList</code> .
	<code>buildStatementList(root)</code>	Membangun <code>CompoundNode</code> dari <code>STATEMENT_LIST</code> dengan mengiterasi semua anak <code>STATEMENT</code> dan memanggil <code>buildStmtNode</code> untuk masing-masing.
	<code>buildProgramNode(root)</code>	Membangun <code>ProgramNode</code> dari root parse tree. Mengekstrak nama program dari <code>PROGRAM_HEADER</code> , deklarasi dari <code>DECLARATION_PART</code> , dan body utama dari <code>COMPOUND_STATEMENT</code> .
SemanticAnalyzer	<code>analyzeAndOutput(root, filename)</code>	Method publik utama untuk memulai analisis semantik. Memanggil <code>visit(root)</code> , lalu mengeluarkan symbol table dan decorated AST ke file output.
	<code>visit(node)</code>	Dispatcher utama untuk semua <code>ASTNode</code> non-ekspresi. Mendispatch ke visitor yang tepat berdasarkan <code>ASTType</code> node.
	<code>visitExpr(node)</code>	Dispatcher untuk semua <code>ExprNode</code> . Mengembalikan kode tipe integer (<code>T_INTEGER</code> , <code>T_REAL</code> , dst.) dan menyimpannya di cache <code>nodeType</code> .
	<code>visitType(node)</code>	Dispatcher untuk <code>TypeNode</code> . Mendelegasikan ke <code>visitSimpleType</code> , <code>visitArrayType</code> , <code>visitEnumType</code> , <code>visitRangeType</code> , atau <code>visitFieldType</code> .
	<code>visitProgram(node)</code>	Mendaftarkan nama program ke symbol table sebagai <code>OBJ_PROC</code> , lalu mengunjungi semua deklarasi dan blok utama.
	<code>visitConstDecl(node)</code>	Memvalidasi tidak ada redeclaration, menginfer tipe dari ekspresi nilai, dan

		mendaftarkan konstanta ke symbol table dengan const_value.
	visitTypeDecl(node)	Memvalidasi tidak ada redeclaration, menginfer tipe, dan mendaftarkan alias tipe sebagai OBJ_TYPE ke symbol table.
	visitVarDecl(node)	Memvalidasi tidak ada redeclaration per variabel, menginfer tipe, dan mendaftarkan setiap variabel ke symbol table dengan alamat (vsze).
	visitParamDecl(node)	Memvalidasi tidak ada duplikasi nama parameter, menginfer tipe, dan mendaftarkan parameter ke symbol table dengan nrm=0 (var-param) atau 1 (value-param).
	visitProcDecl(node)	Mendaftarkan prosedur ke scope luar, membuka scope baru, mengunjungi parameter dan deklarasi lokal, mengunjungi body, lalu menutup scope.
	visitFuncDecl(node)	Mendaftarkan fungsi ke scope luar dengan tipe kembalian, membuka scope baru, mengunjungi parameter dan body, lalu menutup scope.
	visitAssign(node)	Menginfer tipe LHS dan RHS, lalu mengecek kompatibilitas penugasan menggunakan isAssignmentCompatible.
	visitIf(node)	Memvalidasi kondisi bertipe Boolean, lalu mengunjungi blok then dan (jika ada) blok else.
	visitWhile(node)	Memvalidasi kondisi bertipe Boolean, lalu mengunjungi body loop.
	visitFor(node)	Memvalidasi variabel kontrol terdeclare, lalu mengunjungi ekspresi awal, akhir, dan body loop.
	visitRepeat(node)	Mengunjungi body loop, lalu memvalidasi kondisi until bertipe Boolean.
	visitCase(node)	Memvalidasi kunci ekspresi bertipe ordinal (Integer/Char/Boolean), memvalidasi

		kompatibilitas setiap label dengan tipe kunci, lalu mengunjungi setiap statement.
	<code>visitCompound(node)</code>	Mengunjungi setiap statement dalam CompoundNode secara berurutan.
	<code>visitCallStmt(node)</code>	Wrapper yang mendelegasikan ke <code>visitCall</code> untuk CallStmtNode.
	<code>visitLiteral(node)</code>	Mengembalikan kode tipe berdasarkan LiteralKind (Int→T_INTEGER, Real→T_REAL, Char→T_CHAR, String→T_STRING, Bool→T_BOOLEAN).
	<code>visitVarRef(node)</code>	Mencari nama variabel di symbol table; mengecek tidak digunakan sebagai tipe. Mengembalikan tipe dari entri symbol table.
	<code>visitCall(node)</code>	Mencari nama fungsi/prosedur di symbol table, memvalidasi OBJ_FUNC/OBJ_PROC, mengunjungi argumen, dan mengembalikan tipe kembalian.
	<code>visitBinaryOp(node)</code>	Menginfer tipe kedua operan, lalu mengecek kesesuaian operator: relasional → Boolean; and/or → kedua harus Boolean; aritmetika → harus numerik.
	<code>visitUnaryOp(node)</code>	Mengecek operator unary: not → operan harus Boolean; minus/plus → operan harus numerik. Mengembalikan tipe yang sesuai.
	<code>visitArrayAccess(node)</code>	Mengecek bahwa array bertipe T_ARRAY, mengunjungi ekspresi indeks, dan mengembalikan tipe elemen array dari atab.
	<code>visitFieldAccess(node)</code>	Mengecek bahwa record bertipe T_RECORD, mencari nama field di atab, dan mengembalikan tipe field yang diakses.
	<code>visitSimpleType(node)</code>	Mencari nama tipe di symbol table dan mengembalikan kode tipenya; menyimpan ref di lastTypeRef untuk tipe yang bukan builtin.

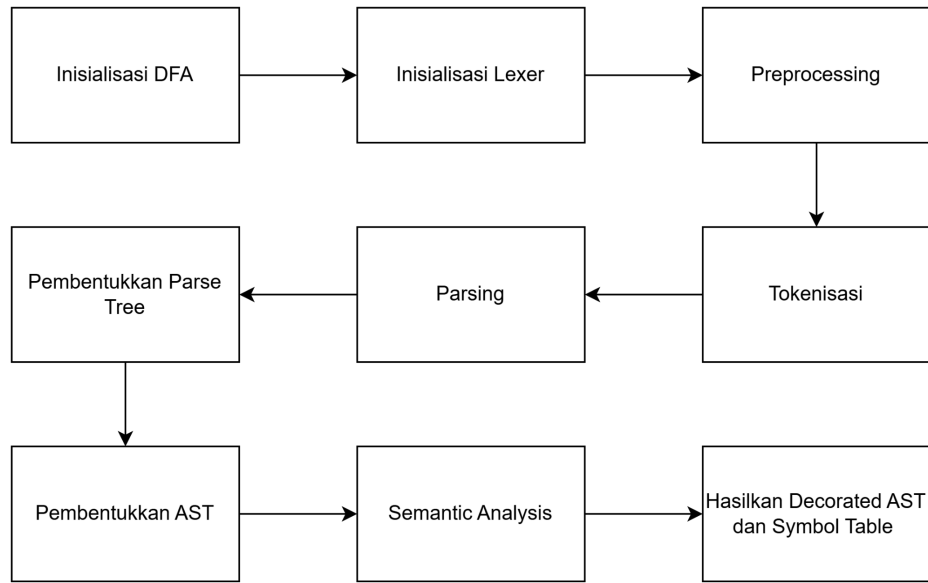
<code>visitArrayType(nod e)</code>	Mengunjungi tipe indeks dan elemen array, mendaftarkan entri baru di atab, dan mengembalikan T_ARRAY.
<code>visitEnumType(node)</code>	Mendaftarkan setiap nilai enumerasi sebagai OBJ_CONST dengan tipe T_INTEGER dan mengembalikan T_INTEGER.
<code>visitRangeType(nod e)</code>	Mengunjungi ekspresi batas bawah dan atas; mengembalikan tipe batas bawah sebagai tipe range.
<code>visitFieldType(nod e)</code>	Mengunjungi setiap field record, mendaftarkan field ke atab, dan mengembalikan T_RECORD.
<code>isCompatible(t1, t2)</code>	Mengecek apakah dua tipe kompatibel untuk operasi biner. Integer dan Real dianggap kompatibel satu sama lain.
<code>isAssignmentCompat ible(target, value)</code>	Mengecek kompatibilitas penugasan, memperbolehkan Integer ke Real promotion.
<code>typeToString(code)</code>	Mengonversi kode tipe integer menjadi string yang dapat dibaca ('integer', 'real', 'boolean', dst.).
<code>printDecoratedAST(root, out)</code>	Mencetak AST yang sudah didekorasi dengan informasi symbol table ke output stream, dimulai dari ProgramNode.
<code>printNode(node, out, prefix, isLast)</code>	Mencetak satu DeclNode dengan anotasi dari symbol table (tab_idx, type, lev, dll.) menggunakan karakter pohon Unicode ($\vdash$, $\sqsubset$, $\mid$).
<code>printExpr(node, out, prefix, isLast)</code>	Mencetak satu ExprNode dengan anotasi tipe yang dievaluasi (misalnya 'integer', 'boolean') menggunakan format pohon.
<code>printStmt(node, out, prefix, isLast)</code>	Mencetak satu StmtNode beserta anak-anaknya secara rekursif menggunakan format pohon bercabang.

SymbolTable	initReserved()	Menginisialisasi tabel simbol dengan semua kata kunci (reserved words), tipe dasar (integer, real, boolean, char, string), konstanta (true, false), dan prosedur built-in (readln, writeln). Dipanggil otomatis dari konstruktor.
	addTab(id, obj, type, ref, nrm, adr)	Menambahkan entri baru ke tab[] pada level saat ini. Mengatur link ke entri sebelumnya di blok yang sama, dan memperbarui btab[getCurrentBlock()].last.
	addBlock()	Membuat entri BlockTab baru di btab[] dan mengembalikan indeksinya. Dipanggil oleh enterScope().
	addArray(xtyp, etyp, eref, low, high)	Membuat entri ArrayTab baru di atab[] dengan informasi tipe indeks, tipe elemen, batas, dan ukuran. Mengembalikan indeks entri.
	searchTab(id)	Mencari identifier dari level saat ini ke atas (melalui display[]) hingga level 0, lalu ke predefined. Mengembalikan indeks atau -1 jika tidak ditemukan.
	searchCurrentScope(id)	Mencari identifier hanya di blok saat ini (btab[getCurrentBlock()]). Digunakan untuk mendeteksi redeclaration.
	enterScope()	Membuat blok baru (addBlock()), menaikkan level, dan memperbarui display[level] ke indeks blok baru.
	exitScope()	Mengosongkan display[level] dan menurunkan level. Entri lokal tetap di tab[] tetapi tidak lagi dapat dicari.
ASTNode	getASTType()	Base class semua node AST. Menyimpan nodeType (enum ASTType) untuk dispatch visitor.
ExprNode	setEvaluatedType / getEvaluatedType	Subclass ASTNode untuk ekspresi. Menyimpan tipe yang dievaluasi sebagai string, indeks symbol table (tabIndex), dan lexical level (lexLevel).

TypeNode	getKind()	Subclass ASTNode untuk tipe. Menyimpan Kind (Simple/Array/Record/Range/Enumerated) untuk dispatch.
LiteralNode	getKind(), getValue()	Literal konstanta; menyimpan LiteralKind dan nilai string.
SimpleTypeNode	getName()	Nama tipe sederhana (misalnya 'integer', 'myType').
ArrayTypeNode	getType(), getIndex()	Tipe array; menyimpan tipe elemen dan tipe indeks (SimpleType atau RangeType).
RangeTypeNode	getLow(), getHigh()	Tipe range; menyimpan ekspresi batas bawah dan atas sebagai ExprNode.
EnumTypeNode	getEnumValues()	Tipe enumerasi; menyimpan daftar nama nilai.
FieldTypeNode	getFields()	Tipe record; menyimpan vektor pasangan (nama-nama, TypeNode) untuk setiap field.
ConstDeclNode	getName(), getValue()	Deklarasi konstanta; nama dan ekspresi nilai.
TypeDeclNode	getName(), getTypeSpec()	Deklarasi alias tipe; nama dan spesifikasi tipe.
VarDeclNode	getNames(), getType()	Deklarasi variabel; daftar nama dan tipe.
ParamDeclNode	getNames(), getType(), isVarParam()	Deklarasi parameter; daftar nama, tipe, dan flag var-param.
ProcDeclNode	getName(), getParams(), getLocalVar(), getBody()	Deklarasi prosedur; nama, parameter, deklarasi lokal, dan body.
FuncDeclNode	getName(), getReturnType(), getParams(), getLocalVar(), getBody()	Deklarasi fungsi; sama dengan ProcDeclNode dengan tambahan tipe kembalian.
BinaryOpNode	getOp(), getLeft(), getRight()	Ekspresi biner; operator (string), operan kiri dan kanan.

UnaryOpNode	getOp(), getOperand()	Ekspresi unary; operator dan operand.
VarRefNode	getName()	Referensi variabel sederhana; nama identifier.
ArrayAccessNode	getArray(), getIndex()	Akses elemen array; ekspresi array dan ekspresi indeks.
FieldAccessNode	getRecord(), getFieldName()	Akses field record; ekspresi record dan nama field.
CallNode	getName(), getArgs()	Pemanggilan fungsi sebagai ekspresi; nama dan daftar argumen.
CallStmtNode	getCall()	Wrapper CallNode sebagai statement.
AssignNode	getTarget(), getValue()	Pernyataan assignment; LHS dan RHS sebagai ExprNode.
IfNode	getCondition(), getThenBlock(), getElseBlock()	Pernyataan if-then-else; kondisi, blok then, dan (opsional) blok else.
WhileNode	getCondition(), getBody()	Pernyataan while; kondisi dan body.
ForNode	getMovingVar(), getStartPoint(), getEndPoint(), getBody(), goesUp()	Pernyataan for; variabel kontrol, nilai awal/akhir, body, dan arah.
RepeatNode	getBody(), getUntilCondition())	Pernyataan repeat-until; body dan kondisi akhir.
CaseNode	getKey(), getCases()	Pernyataan case; kunci ekspresi dan vektor pasangan (label-list, statement).
CompoundNode	getStatements()	Blok begin-end; vektor statement.
ProgramNode	getName(), getDeclarations(), getMain()	Root AST; nama program, deklarasi, dan blok utama.

3.10. Alur Kerja Program



Gambar 3.10.1 Alur Kerja Program

Secara umum, program bekerja melalui tiga tahap utama, yaitu lexical analysis, syntax analysis, dan semantic analysis. Pada tahap lexical analysis, source code diubah menjadi token menggunakan DFA. Selanjutnya, parser melakukan recursive descent parsing untuk membentuk parse tree berdasarkan grammar bahasa. Setelah parse tree berhasil dibentuk, semantic analyzer akan mengonversi parse tree menjadi Abstract Syntax Tree (AST), kemudian melakukan semantic checking untuk menghasilkan Decorated AST beserta Symbol Table. Komponen utama yang digunakan meliputi DFA, Lexer, Parser, AST Builder, Semantic Analyzer, serta Symbol Table.

Berikut penjelasan alur program secara keseluruhan:

1. Inisialisasi DFA

Pada tahap ini, program membaca file aturan DFA (dfa_rules.txt). Semua aturan transisi akan dimuat ke dalam struktur transition.

2. Inisialisasi Lexer

Pada tahap ini, lexer membuka file source code, kemudian membaca karakter per karakter.

3. Preprocessing

Lexer akan melewati spasi, tab, newline, sekaligus melakukan update nomor baris. Dilakukan supaya DFA hanya memproses karakter yang relevan.

4. Proses Tokenisasi

Merupakan loop utama dari program. Menggunakan fungsi `tokenize()` yang didefinisikan di `lexer.cpp`. Dengan alur sebagai berikut:

a) Mulai dari

`currentState = start_state`

`lexeme = ""`

b) Untuk setiap karakter,

Ambil next state:

`nextState = getNextState(...)`

c) Terdapat tiga kemungkinan kasus:

CASE 1: Transisi Valid

Lanjut membangun token. Ini mencakup transisi spesifik dan wildcard.

`lexeme += currentChar;`

`currentState = nextState;`

CASE 2: Masuk Dead State

Terjadi apabila tidak ada transisi spesifik dan tidak ada wildcard.

`if(DFA.isFinalState(currentState))`

o Jika sebelumnya final:

`simpan token`

`reset state`

o Jika tidak:

`error / stop`

d) EOF Handling

Jika masih ada lexeme dan state final, maka tetap jadi token.

e) Output

Menghasilkan `vector<Token>`.

5. Parsing

Setelah proses tokenisasi selesai, vector token dikirim ke parser. Parser menggunakan metode recursive descent parsing untuk memeriksa kesesuaian token terhadap grammar bahasa. Apabila ditemukan token yang tidak sesuai dengan grammar, parser akan mengembalikan `nullptr` dan menampilkan informasi syntax error berupa token yang diharapkan, token yang diterima, serta nomor baris error.

6. Pembentukan Parse Tree

Selama proses parsing, parser membangun struktur parse tree menggunakan `TreeNode`. Setiap non-terminal grammar direpresentasikan sebagai node yang memiliki child sesuai hasil parsing produksi grammar. Parse tree hanya akan dihasilkan apabila seluruh proses parsing berhasil dan source code valid secara sintaks.

7. Pembentukan AST

Program melakukan konversi parse tree menjadi Abstract Syntax Tree (AST) menggunakan pendekatan Syntax-Directed Translation (SDT). Dengan cara, pada setiap aturan produksi grammar, ditambahkan semantic action yang bertugas membangun node AST yang bersesuaian. Selama proses ini, elemen-elemen sintaksis yang tidak lagi diperlukan untuk analisis semantik dihilangkan. Sehingga, hasilnya adalah struktur pohon yang lebih ringkas dan berfokus pada struktur semantik program (AST).

8. Semantic Analysis

Akan dilakukan traversal pada AST yang telah terbentuk secara DFS menggunakan fungsi visitor yang akan melakukan pengecekan semantic program. Di antaranya: lookup identifier pada Symbol Table, pengecekan deklarasi variabel, pengecekan scope, inferensi tipe data, validasi kompatibilitas tipe, serta anotasi semantic pada node AST.

9. Hasilkan Decorated AST dan Symbol Table

Hasil akhir program berupa Decorated Abstract Syntax Tree (DAST), Symbol Table (tab, atab, btab), serta informasi semantic error apabila ditemukan pelanggaran aturan semantic program.

3.11. Pendekatan Implementasi

Implementasi milestone ini terbagi menjadi dua tahap utama, yaitu: pembangunan AST dari parse tree (ASTer) dan analisis semantik beserta pembangunan symbol table (SemanticAnalyzer + SymbolTable).

Pada tahap pertama, pembangunan AST, dirancang dengan pola builder rekursif top-down yang memisahkan logika transformasi dari representasi data. Setiap jenis node pada parse tree dipetakan ke satu method buildXxx() yang bertanggung jawab penuh atas ekstraksi informasi relevan dan pembuangan node-node sintaktis yang tidak bermakna secara semantik (seperti keyword begin/end, titik koma, dan tanda kurung). Pemisahan ini menjaga kode tetap modular: perubahan pada satu jenis statement atau ekspresi tidak berdampak pada builder lainnya. Untuk ekspresi yang memiliki hierarki presedensi (EXPRESSION -> SIMPLE_EXPRESSION -> TERM -> FACTOR), buildExprNode() menyusun rantai BinaryOpNode secara iteratif dari kiri ke kanan sehingga struktur pohon yang dihasilkan sudah mencerminkan presedensi operator secara implisit tanpa memerlukan anotasi tambahan. Akses variabel bertingkat seperti a[i].field juga direpresentasikan sebagai rantai ArrayAccessNode dan FieldAccessNode yang dibangun secara berulang, sehingga setiap level akses terwakili eksplisit dalam AST. Seluruh node AST menggunakan std::shared_ptr untuk manajemen memori otomatis, konsisten dengan pendekatan yang sudah digunakan pada parse tree di milestone sebelumnya.

Lalu di tahap kedua, analisis semantik, menggunakan pola *visitor* yang melakukan penelusuran AST berdasarkan ASTType setiap node. Pilihan pola visitor memungkinkan penambahan pemeriksaan baru (misalnya pemeriksaan jenis baru) tanpa mengubah struktur kelas AST itu sendiri. Tipe yang diperoleh dari pengecekan setiap ekspresi disimpan di cache (`unordered_map<void*, int>`) menggunakan pointer node sebagai kunci, sehingga tipe yang sudah dihitung tidak perlu dihitung ulang jika node yang sama direferensikan lebih dari sekali.

Symbol table diimplementasikan mengikuti desain tiga tabel paralel (`tab[]`, `btab[]`, `atab[]`) yang merupakan pendekatan dari compiler yang paling umum. Pendekatan ini dipilih karena efisien secara memori dan sesuai dengan kebutuhan bahasa Arion yang memiliki scoping leksikal bertingkat: mekanisme `display[]` memetakan lexical level ke blok aktif sehingga pencarian simbol dari scope terdalam ke terluar dapat dilakukan secara linear tanpa traversal pohon scope. Pendeteksian redeclaration menggunakan `searchCurrentScope()` yang hanya memeriksa blok saat ini, sehingga shadowing antar scope yang berbeda tetap diperbolehkan sesuai semantik.

Keluaran akhir berupa *decorated AST* yang menggabungkan struktur pohon dengan anotasi dari symbol table (indeks tabel, tipe, lexical level, nilai konstanta). Representasi ini dipilih karena memudahkan verifikasi kebenaran analisis secara visual: setiap node dapat langsung dicocokkan dengan entri symbol table yang bersesuaian, tanpa harus membaca kedua output secara terpisah.

BAB 4

HASIL IMPLEMENTASI

Class Node AST:

```
enum class LiteralKind { Int, Real, Bool, Char, String };

class ASTNode {
protected:
    ASTType nodeType;
public:
    ASTType getASTType() { return nodeType; }
    virtual ~ASTNode() = default;
};

class ExprNode : public ASTNode {
public:
    void setEvaluatedType(std::string type) { evaluatedType = type; }
    std::string getEvaluatedType() const { return evaluatedType; }

    void setTabIndex(int index) { tabIndex = index; }
    int getTabIndex() const { return tabIndex; }

    void setLexicalLevel(int level) { lexLevel = level; }
    int getLexicalLevel() const { return lexLevel; }

protected:
    std::string evaluatedType = "unknown";
    int tabIndex = -1;
    int lexLevel = -1;
};

class LiteralNode : public ExprNode {
private:
    LiteralKind kind;
    std::string value;
public:
    LiteralNode(LiteralKind kind, std::string value) : kind(kind),
value(value) { nodeType = ASTType::LiteralNode; }
```

```

    LiteralKind getKind(){ return kind; }
    std::string getValue(){ return value; }
};

class TypeNode : public ASTNode {
public:
    enum class Kind { Simple, Array, Record, Range, Enumerated };
    TypeNode(Kind kind) : kind(kind) {}
    Kind getKind() const { return kind; }
private:
    Kind kind;
};

class SimpleTypeNode : public TypeNode {
public:
    SimpleTypeNode(std::string name) : TypeNode(Kind::Simple), name(name)
{ nodeType = ASTType::SimpleTypeNode; }
    std::string getName() const { return name; }
private:
    std::string name;
};

class ArrayTypeNode : public TypeNode {
public:
    ArrayTypeNode(std::shared_ptr<TypeNode> type,
std::shared_ptr<TypeNode> index)
        : TypeNode(Kind::Array), type(type), index(index) { nodeType =
ASTType::ArrayTypeNode; }
    std::shared_ptr<TypeNode> getType() const { return type; }
    std::shared_ptr<TypeNode> getIndex() const { return index; }
private:
    std::shared_ptr<TypeNode> type;
    std::shared_ptr<TypeNode> index; // harus simpletype atau rangetype
};

class EnumTypeNode : public TypeNode {
public:
    EnumTypeNode(std::vector<std::string> enum_values) :
TypeNode(Kind::Enumerated), enum_values(enum_values) { nodeType =
ASTType::EnumTypeNode; }

```

```

        std::vector<std::string> getEnumValues() const { return enum_values; }
        void pushEnumValue(std::string value) { enum_values.push_back(value); }
    }
private:
    std::vector<std::string> enum_values;
};

class RangeTypeNode : public TypeNode {
public:
    RangeTypeNode(std::shared_ptr<ExprNode> low, std::shared_ptr<ExprNode>
high) : TypeNode(Kind::Range), low(low), high(high) { nodeType =
ASTType::RangeTypeNode; }
    std::shared_ptr<ExprNode> getLow() const { return low; }
    std::shared_ptr<ExprNode> getHigh() const { return high; }
private:
    std::shared_ptr<ExprNode> low;
    std::shared_ptr<ExprNode> high;
};

class FieldTypeNode : public TypeNode {
public:
    FieldTypeNode(std::vector<std::pair<std::vector<std::string>,
std::shared_ptr<TypeNode>>> fields)
        : TypeNode(Kind::Record), fields(fields) { nodeType =
ASTType::FieldTypeNode; }
    std::vector<std::pair<std::vector<std::string>,
std::shared_ptr<TypeNode>>> getFields() const { return fields; }
    void pushField(std::pair<std::vector<std::string>,
std::shared_ptr<TypeNode>> field) { fields.push_back(field); }
private:
    std::vector<std::pair<std::vector<std::string>,
std::shared_ptr<TypeNode>>> fields;
};

class StmtNode : public ASTNode {};

class DeclNode : public ASTNode {
public:
    enum class Kind { Const, Type, Var, Proc, Func, Param };
    DeclNode(Kind kind) : kind(kind) {}
};

```

```

    Kind getKind() const { return kind; }
private:
    Kind kind;
};

class TypeDeclNode : public DeclNode {
public:
    TypeDeclNode(std::string name, std::shared_ptr<TypeNode> type_spec)
        : DeclNode(Kind::Type), name(name), type_spec(type_spec) {
nodeType = ASTType::TypeDeclNode; }
    std::string getName() const { return name; }
    std::shared_ptr<TypeNode> getTypeSpec() const { return type_spec; }
private:
    std::string name;
    std::shared_ptr<TypeNode> type_spec;
};

class ConstDeclNode : public DeclNode {
public:
    ConstDeclNode(std::string name, std::shared_ptr<ExprNode> value)
        : DeclNode(Kind::Const), name(name), value(value) { nodeType =
ASTType::ConstDeclNode; }
    std::string getName() const { return name; }
    std::shared_ptr<ExprNode> getValue() const { return value; }
private:
    std::string name;
    std::shared_ptr<ExprNode> value;
};

class VarDeclNode : public DeclNode {
public:
    VarDeclNode(std::vector<std::string> names, std::shared_ptr<TypeNode>
type)
        : DeclNode(Kind::Var), names(names), type(type) { nodeType =
ASTType::VarDeclNode; }
    std::vector<std::string> getNames() const { return names; }
    std::shared_ptr<TypeNode> getType() const { return type; }
    void pushName(std::string name) { names.push_back(name); }
private:
    std::vector<std::string> names;

```

```

        std::shared_ptr<TypeNode> type;
};

class ParamDeclNode : public DeclNode {
public:
    ParamDeclNode(std::vector<std::string> names,
std::shared_ptr<TypeNode> type, bool is_var_param = false)
        : DeclNode(Kind::Param), names(names), type(type),
is_var_param(is_var_param) { nodeType = ASTType::ParamDeclNode; }
    std::vector<std::string> getNames() const { return names; }
    std::shared_ptr<TypeNode> getType() const { return type; }
    bool isVarParam() const { return is_var_param; }
    void pushName(std::string name) { names.push_back(name); }
private:
    std::vector<std::string> names;
    std::shared_ptr<TypeNode> type;
    bool is_var_param;
};

class ProcDeclNode : public DeclNode {
public:
    ProcDeclNode(std::string name,
std::vector<std::shared_ptr<ParamDeclNode>> params,
        std::vector<std::shared_ptr<DeclNode>> local_var,
std::shared_ptr<StmtNode> body)
        : DeclNode(Kind::Proc), name(name), params(params),
local_var(local_var), body(body) { nodeType = ASTType::ProcDeclNode; }
    std::string getName() const { return name; }
    std::vector<std::shared_ptr<ParamDeclNode>> getParams() const { return
params; }
    std::vector<std::shared_ptr<DeclNode>> getLocalVar() const { return
local_var; }
    std::shared_ptr<StmtNode> getBody() const { return body; }
    void pushParam(std::shared_ptr<ParamDeclNode> param) {
params.push_back(param); }
    void pushLocalVar(std::shared_ptr<DeclNode> var) {
local_var.push_back(var); }
private:
    std::string name;
    std::vector<std::shared_ptr<ParamDeclNode>> params;

```

```

    std::vector<std::shared_ptr<DeclNode>> local_var;
    std::shared_ptr<StmtNode> body;
};

class FuncDeclNode : public DeclNode {
public:
    FuncDeclNode(std::string name, std::shared_ptr<TypeNode> return_type,
                  std::vector<std::shared_ptr<ParamDeclNode>> params,
                  std::vector<std::shared_ptr<DeclNode>> local_var,
std::shared_ptr<StmtNode> body)
        : DeclNode(Kind::Func), name(name), return_type(return_type),
params(params), local_var(local_var), body(body) { nodeType =
ASTType::FuncDeclNode; }
    std::string getName() const { return name; }
    std::shared_ptr<TypeNode> getReturnType() const { return return_type;
}
    std::vector<std::shared_ptr<ParamDeclNode>> getParams() const { return
params; }
    std::vector<std::shared_ptr<DeclNode>> getLocalVar() const { return
local_var; }
    std::shared_ptr<StmtNode> getBody() const { return body; }
    void pushParam(std::shared_ptr<ParamDeclNode> param) {
params.push_back(param); }
    void pushLocalVar(std::shared_ptr<DeclNode> var) {
local_var.push_back(var); }
private:
    std::string name;
    std::shared_ptr<TypeNode> return_type;
    std::vector<std::shared_ptr<ParamDeclNode>> params;
    std::vector<std::shared_ptr<DeclNode>> local_var;
    std::shared_ptr<StmtNode> body;
};

class CallNode : public ExprNode {
public:
    CallNode(std::string name, std::vector<std::shared_ptr<ExprNode>>
args)
        : name(name), args(args) { nodeType = ASTType::CallNode; }
    std::string getName() const { return name; }
    std::vector<std::shared_ptr<ExprNode>> getArgs() const { return args;

```

```

}

    void pushArg(std::shared_ptr<ExprNode> arg) { args.push_back(arg); }
private:
    std::string name;
    std::vector<std::shared_ptr<ExprNode>> args;
};

class CallStmtNode : public StmtNode {
public:
    CallStmtNode(std::shared_ptr<CallNode> call)
        : call(call) { nodeType = ASTType::CallStmtNode; }

    std::shared_ptr<CallNode> getCall() const { return call; }
private:
    std::shared_ptr<CallNode> call;
};

class BinaryOpNode : public ExprNode {
public:
    BinaryOpNode(std::string op, std::shared_ptr<ExprNode> left,
std::shared_ptr<ExprNode> right)
        : op(op), left(left), right(right) { nodeType =
ASTType::BinaryOpNode; }
    std::string getOp() const { return op; }
    std::shared_ptr<ExprNode> getLeft() const { return left; }
    std::shared_ptr<ExprNode> getRight() const { return right; }
private:
    std::string op;
    std::shared_ptr<ExprNode> left;
    std::shared_ptr<ExprNode> right;
};

class UnaryOpNode : public ExprNode {
public:
    UnaryOpNode(std::string op, std::shared_ptr<ExprNode> operand)
        : op(op), operand(operand) { nodeType = ASTType::UnaryOpNode; }
    std::string getOp() const { return op; }
    std::shared_ptr<ExprNode> getOperand() const { return operand; }
private:
    std::string op;

```

```

        std::shared_ptr<ExprNode> operand;
};

class VarRefNode : public ExprNode {
public:
    VarRefNode(std::string name) : name(name) { nodeType =
ASTType::VarRefNode; }
    std::string getName() const { return name; }
private:
    std::string name;
};

class ArrayAccessNode : public ExprNode {
public:
    ArrayAccessNode(std::shared_ptr<ExprNode> array,
std::shared_ptr<ExprNode> index)
        : array(array), index(index) { nodeType =
ASTType::ArrayAccessNode; }
    std::shared_ptr<ExprNode> getArray() const { return array; }
    std::shared_ptr<ExprNode> getIndex() const { return index; }
private:
    std::shared_ptr<ExprNode> array;
    std::shared_ptr<ExprNode> index;
};

class FieldAccessNode : public ExprNode {
public:
    FieldAccessNode(std::shared_ptr<ExprNode> record, std::string
field_name)
        : record(record), field_name(field_name) { nodeType =
ASTType::FieldAccessNode; }
    std::shared_ptr<ExprNode> getRecord() const { return record; }
    std::string getFieldName() const { return field_name; }
private:
    std::shared_ptr<ExprNode> record;
    std::string field_name;
};

class IfNode : public StmtNode {
public:

```

```

        IfNode(std::shared_ptr<ExprNode> condition, std::shared_ptr<StmtNode>
thenblock,
            std::shared_ptr<StmtNode> elseblock = nullptr)
            : condition(condition), thenblock(thenblock), elseblock(elseblock)
{ nodeType = ASTType::IfNode; }
    std::shared_ptr<ExprNode> getCondition() const { return condition; }
    std::shared_ptr<StmtNode> getThenBlock() const { return thenblock; }
    std::shared_ptr<StmtNode> getElseBlock() const { return elseblock; }
private:
    std::shared_ptr<ExprNode> condition;
    std::shared_ptr<StmtNode> thenblock;
    std::shared_ptr<StmtNode> elseblock;
};

class WhileNode : public StmtNode {
public:
    WhileNode(std::shared_ptr<ExprNode> condition,
std::shared_ptr<StmtNode> body)
        : condition(condition), body(body) { nodeType =
ASTType::WhileNode; }
    std::shared_ptr<ExprNode> getCondition() const { return condition; }
    std::shared_ptr<StmtNode> getBody() const { return body; }
private:
    std::shared_ptr<ExprNode> condition;
    std::shared_ptr<StmtNode> body;
};

class ForNode : public StmtNode {
public:
    ForNode(bool goes_up, std::string movingvar, std::shared_ptr<ExprNode>
startpoint,
        std::shared_ptr<ExprNode> endpoint, std::shared_ptr<StmtNode>
body)
        : goes_up(goes_up), movingvar(movingvar), startpoint(startpoint),
endpoint(endpoint), body(body) { nodeType = ASTType::ForNode; }
    bool goesUp() const { return goes_up; }
    std::string getMovingVar() const { return movingvar; }
    std::shared_ptr<ExprNode> getStartPoint() const { return startpoint; }
    std::shared_ptr<ExprNode> getEndPoint() const { return endpoint; }
    std::shared_ptr<StmtNode> getBody() const { return body; }
};

```

```

private:
    bool goes_up;
    std::string movingvar;
    std::shared_ptr<ExprNode> startpoint;
    std::shared_ptr<ExprNode> endpoint;
    std::shared_ptr<StmtNode> body;
};

class RepeatNode : public StmtNode {
public:
    RepeatNode(std::shared_ptr<StmtNode> body, std::shared_ptr<ExprNode>
untilcondition)
        : body(body), untilcondition(untilcondition) { nodeType =
ASTType::RepeatNode; }
    std::shared_ptr<StmtNode> getBody() const { return body; }
    std::shared_ptr<ExprNode> getUntilCondition() const { return
untilcondition; }
private:
    std::shared_ptr<StmtNode> body;
    std::shared_ptr<ExprNode> untilcondition;
};

class CaseNode : public StmtNode {
public:
    CaseNode(std::shared_ptr<ExprNode> key,
        std::vector<std::pair<std::vector<std::shared_ptr<ExprNode>>,
std::shared_ptr<StmtNode>>> cases)
        : key(key), cases(cases) { nodeType = ASTType::CaseNode; }
    std::shared_ptr<ExprNode> getKey() const { return key; }
    std::vector<std::pair<std::vector<std::shared_ptr<ExprNode>>,
std::shared_ptr<StmtNode>>> getCases() const { return cases; }
    void pushCase(std::pair<std::vector<std::shared_ptr<ExprNode>>,
std::shared_ptr<StmtNode>> case_pair) { cases.push_back(case_pair); }
private:
    std::shared_ptr<ExprNode> key;
    std::vector<std::pair<std::vector<std::shared_ptr<ExprNode>>,
std::shared_ptr<StmtNode>>> cases;
};

class AssignNode : public StmtNode {

```

```

public:
    AssignNode(std::shared_ptr<ExprNode> target, std::shared_ptr<ExprNode>
value)
        : target(target), value(value) { nodeType = ASTType::AssignNode; }
    std::shared_ptr<ExprNode> getTarget() const { return target; }
    std::shared_ptr<ExprNode> getValue() const { return value; }
private:
    std::shared_ptr<ExprNode> target;
    std::shared_ptr<ExprNode> value;
};

class CompoundNode : public StmtNode {
public:
    CompoundNode(std::vector<std::shared_ptr<StmtNode>> statements)
        : statements(statements) { nodeType = ASTType::CompoundNode; }
    std::vector<std::shared_ptr<StmtNode>> getStatements() const { return
statements; }
    void pushStatement(std::shared_ptr<StmtNode> stmt) {
statements.push_back(stmt); }
private:
    std::vector<std::shared_ptr<StmtNode>> statements;
};

class ProgramNode : public ASTNode {
public:
    ProgramNode(std::string name, std::vector<std::shared_ptr<DeclNode>>
declarations,
                std::shared_ptr<CompoundNode> main)
        : name(name), declarations(declarations), main(main) { nodeType =
ASTType::ProgramNode; }
    std::string getName() const { return name; }
    std::vector<std::shared_ptr<DeclNode>> getDeclarations() const {
return declarations; }
    std::shared_ptr<CompoundNode> getMain() const { return main; }
    void pushDeclaration(std::shared_ptr<DeclNode> decl) {
declarations.push_back(decl); }
private:
    std::string name;
    std::vector<std::shared_ptr<DeclNode>> declarations;
    std::shared_ptr<CompoundNode> main;

```

```
};
```

Penjelasan:

Setiap kelas dalam AST.hpp memiliki satu kelas abstrak utama, yaitu ASTNode. Setiap node menyimpan atribut nodeType bertipe ASTType yang mengidentifikasi jenis node tersebut (seperti LiteralNode, AssignNode, ProgramNode, dsb). Kelas ini memiliki destruktur virtual sehingga seluruh hierarki turunannya bersifat polimorfik.

Dari ASTNode, terdapat tiga kelas turunan utama: ExprNode untuk node yang menghasilkan nilai (ekspresi), StmtNode untuk node yang merepresentasikan aksi (pernyataan), dan DeclNode untuk node yang mendefinisikan nama baru (deklarasi). TypeNode juga merupakan turunan langsung ASTNode yang merepresentasikan sistem tipe. Pembagian ini mencerminkan struktur grammar bahasa Arion yang digunakan.

ExprNode memiliki atribut tambahan yaitu evaluatedType, tabIndex, dan lexLevel yang diisi saat fase analisis semantik. Atribut-atribut ini menghubungkan setiap ekspresi ke entri dalam symbol table, sehingga informasi tipe dan scope dapat diakses selama proses kompilasi selanjutnya seperti pengecekan tipe dan pembangkitan kode.

Class ASTer:

```

1  #pragma once
2
3  #include "AST.hpp"
4  #include "../Parser/ParseTree.hpp"
5
6  class ASTer {
7  public:
8      ASTer(){}
9      std::shared_ptr<ASTNode> buildAST(std::shared_ptr<TreeNode> root) const;
10
11      std::pair<std::vector<std::string>, std::shared_ptr<TypeNode>>
12          fieldPartToField(std::shared_ptr<TreeNode> node) const;
13      std::shared_ptr<TypeNode> buildSimpleType(std::shared_ptr<TreeNode> node) const;
14      std::shared_ptr<TypeNode> buildArrayType(std::shared_ptr<TreeNode> node) const;
15      std::shared_ptr<TypeNode> buildRangeType(std::shared_ptr<TreeNode> node) const;
16      std::shared_ptr<TypeNode> buildEnumType(std::shared_ptr<TreeNode> node) const;
17      std::shared_ptr<TypeNode> buildRecordType(std::shared_ptr<TreeNode> node) const;
18      std::shared_ptr<TypeNode> buildArrayIndex(std::shared_ptr<TreeNode> node) const;
19      std::shared_ptr<TypeNode> buildTypeNode(std::shared_ptr<TreeNode> root) const;
20
21      std::vector<std::shared_ptr<DeclNode>> buildDeclNodes(std::shared_ptr<TreeNode> root) const;
22      std::vector<std::shared_ptr<ConstDeclNode>> buildConstDeclNode(std::shared_ptr<TreeNode> root) const;
23      std::vector<std::shared_ptr<TypeDeclNode>> buildTypeDeclNode(std::shared_ptr<TreeNode> root) const;
24      std::vector<std::shared_ptr<VarDeclNode>> buildVarDeclNode(std::shared_ptr<TreeNode> root) const;
25      std::shared_ptr<ParamDeclNode> buildParamDeclNode(std::shared_ptr<TreeNode> root) const;
26      std::vector<std::shared_ptr<ParamDeclNode>> buildFormalParams(std::shared_ptr<TreeNode> root) const;
27      std::shared_ptr<ProcDeclNode> buildProcDeclNode(std::shared_ptr<TreeNode> root) const;
28      std::shared_ptr<FuncDeclNode> buildFuncDeclNode(std::shared_ptr<TreeNode> root) const;
29
30      std::shared_ptr<LiteralNode> buildLiteralNode(std::shared_ptr<TreeNode> root) const;
31      std::shared_ptr<CallNode> buildCallNode(std::shared_ptr<TreeNode> root) const;
32      std::shared_ptr<BinaryOpNode> buildBinaryOpNode(std::shared_ptr<ExprNode> lhs,
33          std::shared_ptr<TreeNode> opNode, std::shared_ptr<ExprNode> rhs) const;
34      std::shared_ptr<UnaryOpNode> buildUnaryOpNode(std::shared_ptr<TreeNode> opNode,
35          std::shared_ptr<ExprNode> operand) const;
36
37      std::shared_ptr<VarRefNode> buildVarRefNode(std::shared_ptr<TreeNode> root) const;
38      std::shared_ptr<ArrayAccessNode> buildArrayAccessNode(std::shared_ptr<TreeNode> root) const;
39      std::shared_ptr<FieldAccessNode> buildFieldAccessNode(std::shared_ptr<TreeNode> root) const;
40      std::shared_ptr<AssignNode> buildAssignNode(std::shared_ptr<TreeNode> root) const;
41
42      std::shared_ptr<IfNode> buildIfNode(std::shared_ptr<TreeNode> root) const;
43      std::shared_ptr<ExprNode> buildExprNode(std::shared_ptr<TreeNode> root) const;
44      std::shared_ptr<StmtNode> buildStmtNode(std::shared_ptr<TreeNode> root) const;
45      std::shared_ptr<WhileNode> buildWhileNode(std::shared_ptr<TreeNode> root) const;
46      std::shared_ptr<ForNode> buildForNode(std::shared_ptr<TreeNode> root) const;
47      std::shared_ptr<RepeatNode> buildRepeatNode(std::shared_ptr<TreeNode> root) const;
48      std::shared_ptr<CaseNode> buildCaseNode(std::shared_ptr<TreeNode> root) const;
49      std::shared_ptr<CompoundNode> buildCompoundNode(std::shared_ptr<TreeNode> root) const;
50      std::shared_ptr<ProgramNode> buildProgramNode(std::shared_ptr<TreeNode> root) const;
51      std::shared_ptr<CompoundNode> buildStatementList(std::shared_ptr<TreeNode> root) const;
52  };

```

Penjelasan:

Kelas ASTer memiliki peran utama sebagai *converter* dari Parse Tree menjadi AST, setiap metode pada kelas ini digunakan untuk membuat tiap jenis node dalam AST. Kelas ini tidak memiliki atribut, untuk melakukan transformasi *node* dari Parse Tree akan di-pass ke dalam metode. Untuk membentuk *root* AST dari Parse Tree, hanya perlu memanggil `buildAST()` melalui kelas ini.

Class SymbolTable:

```
1  #pragma once
2  #include <vector>
3  #include <string>
4
5  // obj: reserved = 0, konstanta = 1, variabel = 2, tipe = 3, fungsi = 4, prosedur = 5
6  // type: none/procedure = 0, integer = 1, real = 2, boolean = 3, char = 4,
7  //      array = 5, record = 6, string = 7
8  struct Tab{
9      std::string id;
10     int link;
11     int obj;
12     int type;
13     int ref;
14     int nrm;
15     int lev;
16     int adr;
17
18     std::string const_value;
19
20     Tab() : link(0), obj(2), type(0), ref(0), nrm(1), lev(0), adr(0) {}
21 };
22
23 struct ArrayTab{
24     int arrays;
25     int xtyp;
26     int etyp;
27     int eref;
28     int low;
29     int high;
30     int elsz;
31     int size;
32
33     ArrayTab() : arrays(0), xtyp(1), etyp(1), eref(0), low(0), high(0), elsz(1), size(0) {}
34 };
35
36 struct BlockTab{
37     int blocks;
38     int last;
39     int lpar;
40     int psze;
41     int vsze;
42
43     BlockTab() : blocks(0), last(0), lpar(0), psze(0), vsze(0) {}
44 };
```

```

1  class SymbolTable {
2      private:
3          std::vector<Tab> tab;
4          std::vector<ArrayTab> atab;
5          std::vector<BlockTab> btab;
6          std::vector<int> display;
7          int level;
8          int predefinedCount;
9
10         void initReserved();
11
12     public:
13         SymbolTable() : level(0), predefinedCount(0){
14             initReserved();
15         }
16         int addTab(const std::string& identifiers, int obj, int type, int ref = 0, int nrm = 1, int adr = 0);
17         int addBlock();
18         int addArray(int xtyp, int etyp, int eref, int low, int high);
19         int searchTab(const std::string& identifiers);
20         int searchCurrentScope(const std::string& identifiers);
21
22         void enterScope();
23         void exitScope();
24
25         Tab& getTab(int index){return tab[index];}
26         BlockTab& getBlockTab(int index){return btab[index];}
27         ArrayTab& getArrayTab(int index){return atab[index];}
28
29         int getCurrentLevel() {return level;}
30         int getCurrentBlock() {return display[level];}
31         int getTabSize() {return tab.size();}
32         int getBlocktabSize() {return btab.size();}
33         int getArraytabSize() {return atab.size();}
34         const std::vector<int>& getDisplay() {return display;}
35
36         void setBlockLast(int blockIdx, int lastIdx);
37         void setBlockLpar(int blockIdx, int lparIdx);
38         void setBlockSizes(int blockIdx, int psze, int vsze);
39
40         std::string toString() const;
41         void outputTable(std::ostream& out) const;
42 };

```

Penjelasan:

Kelas ini merupakan representasi dari *symbol table*, *array table*, serta *block table* di dalam *compiler*. Setiap baris pada tabel diimplementasikan sebagai sebuah *struct* untuk menyimpan informasi-informasi terkait. Pemanggilan **constructor** pada kelas ini berfungsi untuk memulai sebuah tabel kosong yang akan diisi oleh kelas lain. Selain metode-metode getter/setter, terdapat juga metode `toString()` dan `outputTable()` yang berfungsi untuk mengubah tabel yang terbentuk menjadi bentuk *string* yang dapat di-print

Class SemanticAnalyzer:

```

1  #pragma once
2
3  #include <memory>
4  #include <string>
5  #include <vector>
6  #include <iostream>
7  #include <stdexcept>
8  #include <unordered_map>
9  #include "AST.hpp"
10 #include "SymbolTable.hpp"
11
12 class SemanticAnalyzer {
13 private:
14     SymbolTable& symTab;
15     int errorCount;
16     int lastTypeRef;
17
18     static const int T_NONE = 0;
19     static const int T_INTEGER = 1;
20     static const int T_REAL = 2;
21     static const int T_BOOLEAN = 3;
22     static const int T_CHAR = 4;
23     static const int T_ARRAY = 5;
24     static const int T_RECORD = 6;
25     static const int T_STRING = 7;
26
27     static const int OBJ_CONST = 1;
28     static const int OBJ_VAR = 2;
29     static const int OBJ_TYPE = 3;
30     static const int OBJ_FUNC = 4;
31     static const int OBJ_PROC = 5;
32
33     void semanticError(const std::string& message) {
34         std::cerr << "Semantic Error: " << message << std::endl;
35         errorCount++;
36     }
37
38     std::unordered_map<void*, int> nodeType;
39     const std::string BRANCH = "\u251C\u2500 "; // └
40     const std::string CORNER = "\u2514\u2500 "; // ┘
41     const std::string PIPE = "\u2502 "; // |
42     const std::string SPACE = " ";
43     std::string conn(bool isLast) const {
44         if(isLast) return CORNER;
45         return BRANCH;
46     }
47     std::string ext (bool isLast) const {
48         if(isLast) return SPACE;
49         return PIPE;
50     }

```

```

1  public:
2      SemanticAnalyzer(SymbolTable& st) : lastTypeRef(0), symTab(st), errorCount(0) {}
3      bool analyzeAndOutput(std::shared_ptr<ASTNode> root, std::string filename) {
4          errorCount = 0;
5          if (!root) return false;
6
7          try {
8              visit(root);
9              std::ofstream output(filename);
10             if(!output){
11                 std::cerr << "Failed to open output file: " << filename << std::endl;
12                 return false;
13             }
14             symTab.outputTable(output);
15             printDecoratedAST(root, output);
16         } catch (const std::exception& e) {
17             semanticError("Fatal crash saat semantic analysis: " + std::string(e.what()));
18         }
19
20         return errorCount == 0;
21     }
22
23     int getErrorCount() const { return errorCount; }
24     int getCachedType(void* ptr) const {
25         auto it = nodeType.find(ptr);
26         return it != nodeType.end() ? it->second : T_NONE;
27     }
28     void cacheType(void* ptr, int t) { nodeType[ptr] = t; }
29
30 private:
31     // main visitors
32     void visit(std::shared_ptr<ASTNode> node);
33     int visitExpr(std::shared_ptr<ExprNode> node);
34     int visitType(std::shared_ptr<TypeNode> node);
35
36     void visitProgram(std::shared_ptr<ProgramNode> node);
37     void visitConstDecl(std::shared_ptr<ConstDeclNode> node);
38     void visitTypeDecl(std::shared_ptr<TypeDeclNode> node);
39     void visitVarDecl(std::shared_ptr<VarDeclNode> node);
40     void visitParamDecl(std::shared_ptr<ParamDeclNode> node);
41     void visitProcDecl(std::shared_ptr<ProcDeclNode> node);
42     void visitFuncDecl(std::shared_ptr<FuncDeclNode> node);
43

```

```

1 // statement visitors
2 void visitAssign(std::shared_ptr<AssignNode> node);
3 void visitIf(std::shared_ptr<IfNode> node);
4 void visitWhile(std::shared_ptr<WhileNode> node);
5 void visitFor(std::shared_ptr<ForNode> node);
6 void visitRepeat(std::shared_ptr<RepeatNode> node);
7 void visitCase(std::shared_ptr<CaseNode> node);
8 void visitCompound(std::shared_ptr<CompoundNode> node);
9 void visitCallStmt(std::shared_ptr<CallStmtNode> node);
10
11 // expression visitors
12 int visitLiteral(std::shared_ptr<LiteralNode> node);
13 int visitCall(std::shared_ptr<CallNode> node);
14 int visitBinaryOp(std::shared_ptr<BinaryOpNode> node);
15 int visitUnaryOp(std::shared_ptr<UnaryOpNode> node);
16 int visitVarRef(std::shared_ptr<VarRefNode> node);
17 int visitArrayAccess(std::shared_ptr<ArrayAccessNode> node);
18 int visitFieldAccess(std::shared_ptr<FieldAccessNode> node);
19
20 // type specification visitors
21 int visitSimpleType(std::shared_ptr<SimpleTypeNode> node);
22 int visitArrayType(std::shared_ptr<ArrayTypeNode> node);
23 int visitEnumType(std::shared_ptr<EnumTypeNode> node);
24 int visitRangeType(std::shared_ptr<RangeTypeNode> node);
25 int visitFieldType(std::shared_ptr<FieldTypeNode> node);
26
27 // helpers
28 bool isCompatible(int type1, int type2) const;
29 bool isAssignmentCompatible(int targetType, int valueType) const;
30 std::string typeToString(int typeCode) const;
31 std::string annotateExpr(std::shared_ptr<ExprNode> node) const;
32 void printExpr(std::shared_ptr<ExprNode> node, std::ostream& out, const std::string& prefix, bool isLast) const;
33 void printStmt(std::shared_ptr<StmtNode> node, std::ostream& out, const std::string& prefix, bool isLast) const;
34 void printNode(std::shared_ptr<ASTNode> node, std::ostream& out, const std::string& prefix, bool isLast) const;
35 void printDecoratedAST(std::shared_ptr<ASTNode> root, std::ostream& out) const;
36 };

```

Penjelasan:

Kelas `SemanticAnalyzer` bertugas untuk melakukan analisis semantik terhadap AST yang telah dibangun oleh `ASTer`. Kelas ini menggunakan pola **visitor** untuk melintasi setiap node AST secara rekursif, memvalidasi kebenaran semantik program, dan mendaftarkan seluruh identifier ke symbol table. Untuk deklarasi, analyzer memastikan tidak ada redeclaration pada scope yang sama, menginferensikan tipe setiap variabel, konstanta, dan parameter, serta membuka dan menutup scope baru untuk setiap prosedur dan fungsi. Untuk ekspresi, analyzer mengevaluasi tipe setiap node (literal, referensi variabel, operasi biner/unary, pemanggilan fungsi, akses array, dan akses field) lalu menyimpan hasilnya di cache untuk efisiensi. Untuk pernyataan, analyzer memverifikasi kebenaran penggunaan tipe: kondisi pada `if`, `while`, dan `repeat-until` harus bertipe Boolean; kunci pada `case` harus bertipe ordinal; dan tipe pada sisi kanan penugasan harus kompatibel dengan sisi kiri. Setelah seluruh traversal selesai, analyzer menghasilkan output berupa symbol table dan decorated AST yang menggabungkan struktur pohon dengan anotasi tipe dan informasi symbol table pada setiap node.

BAB 5

EKSPERIMEN

❖ Kasus uji 1

Tabel 5.1 Kasus Uji 1

Input																																																																																																																																																																																																																															
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>																																																																																																																																																																																																																															
Output																																																																																																																																																																																																																															
SYMBOL TABLE <table> <thead> <tr> <th>IDX</th><th>ID</th><th>OBJ</th><th>TYPE</th><th>REF</th><th>NRM</th><th>LEV</th><th></th></tr> <tr> <th>ADR</th><th>CONST_VAL</th><th></th><th></th><th></th><th></th><th></th><th></th></tr> </thead> <tbody> <tr><td>0</td><td>and</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>array</td><td>3</td><td>5</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>2</td><td>begin</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>3</td><td>case</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>4</td><td>const</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>5</td><td>div</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>6</td><td>downto</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>7</td><td>do</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>8</td><td>else</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>9</td><td>end</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>10</td><td>for</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>11</td><td>function</td><td>4</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>12</td><td>if</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>13</td><td>mod</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>14</td><td>not</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>15</td><td>of</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>16</td><td>or</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>17</td><td>procedure</td><td>5</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>18</td><td>program</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>19</td><td>record</td><td>3</td><td>6</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>20</td><td>repeat</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>21</td><td>integer</td><td>3</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>22</td><td>real</td><td>3</td><td>2</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>23</td><td>boolean</td><td>3</td><td>3</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>24</td><td>char</td><td>3</td><td>4</td><td>0</td><td>1</td><td>0</td><td>0</td></tr> </tbody> </table>								IDX	ID	OBJ	TYPE	REF	NRM	LEV		ADR	CONST_VAL							0	and	0	0	0	1	0	0	1	array	3	5	0	1	0	0	2	begin	0	0	0	1	0	0	3	case	0	0	0	1	0	0	4	const	0	0	0	1	0	0	5	div	0	0	0	1	0	0	6	downto	0	0	0	1	0	0	7	do	0	0	0	1	0	0	8	else	0	0	0	1	0	0	9	end	0	0	0	1	0	0	10	for	0	0	0	1	0	0	11	function	4	0	0	1	0	0	12	if	0	0	0	1	0	0	13	mod	0	0	0	1	0	0	14	not	0	0	0	1	0	0	15	of	0	0	0	1	0	0	16	or	0	0	0	1	0	0	17	procedure	5	0	0	1	0	0	18	program	0	0	0	1	0	0	19	record	3	6	0	1	0	0	20	repeat	0	0	0	1	0	0	21	integer	3	1	0	1	0	0	22	real	3	2	0	1	0	0	23	boolean	3	3	0	1	0	0	24	char	3	4	0	1	0	0
IDX	ID	OBJ	TYPE	REF	NRM	LEV																																																																																																																																																																																																																									
ADR	CONST_VAL																																																																																																																																																																																																																														
0	and	0	0	0	1	0	0																																																																																																																																																																																																																								
1	array	3	5	0	1	0	0																																																																																																																																																																																																																								
2	begin	0	0	0	1	0	0																																																																																																																																																																																																																								
3	case	0	0	0	1	0	0																																																																																																																																																																																																																								
4	const	0	0	0	1	0	0																																																																																																																																																																																																																								
5	div	0	0	0	1	0	0																																																																																																																																																																																																																								
6	downto	0	0	0	1	0	0																																																																																																																																																																																																																								
7	do	0	0	0	1	0	0																																																																																																																																																																																																																								
8	else	0	0	0	1	0	0																																																																																																																																																																																																																								
9	end	0	0	0	1	0	0																																																																																																																																																																																																																								
10	for	0	0	0	1	0	0																																																																																																																																																																																																																								
11	function	4	0	0	1	0	0																																																																																																																																																																																																																								
12	if	0	0	0	1	0	0																																																																																																																																																																																																																								
13	mod	0	0	0	1	0	0																																																																																																																																																																																																																								
14	not	0	0	0	1	0	0																																																																																																																																																																																																																								
15	of	0	0	0	1	0	0																																																																																																																																																																																																																								
16	or	0	0	0	1	0	0																																																																																																																																																																																																																								
17	procedure	5	0	0	1	0	0																																																																																																																																																																																																																								
18	program	0	0	0	1	0	0																																																																																																																																																																																																																								
19	record	3	6	0	1	0	0																																																																																																																																																																																																																								
20	repeat	0	0	0	1	0	0																																																																																																																																																																																																																								
21	integer	3	1	0	1	0	0																																																																																																																																																																																																																								
22	real	3	2	0	1	0	0																																																																																																																																																																																																																								
23	boolean	3	3	0	1	0	0																																																																																																																																																																																																																								
24	char	3	4	0	1	0	0																																																																																																																																																																																																																								

25	string	3	7	0	1	0	0
26	then	0	0	0	1	0	0
27	to	0	0	0	1	0	0
28	type	0	0	0	1	0	0
29	until	0	0	0	1	0	0
30	var	0	0	0	1	0	0
31	while	0	0	0	1	0	0
32	true	1	3	0	1	0	1
true							
33	false	1	3	0	1	0	0
false							
34	readln	5	0	0	1	0	0
35	writeln	5	0	0	1	0	0
36	Hello	5	0	0	1	0	0
37	a	2	1	0	1	0	0
38	b	2	1	0	1	0	1

BLOCK TABLE

IDX	LAST	LPAR	PSZE	VSZE
0	38	0	0	2

ARRAY TABLE

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
idx	xtyp	etyp	eref	low	high	elsz	size

=== DECORATED ABSTRACT SYNTAX TREE ===

ProgramNode(name: 'Hello')

└─ Declarations

└─ VarDecl('a') → tab_idx:37, type:integer, lev:0

└─ VarDecl('b') → tab_idx:38, type:integer, lev:0

└─ Block (main compound)

└─ Assign → type:void

└─ Var('a') → tab_idx:37, type:integer, lev:0

└─ Literal(5) → type:integer

└─ Assign → type:void

└─ Var('b') → tab_idx:38, type:integer, lev:0

└─ BinaryOp('plus') → type:integer

└─ Var('a') → tab_idx:37, type:integer, lev:0

└─ Literal(10) → type:integer

└─ CallStmt('writeln')

└─ Literal('Result = ') → type:none

└─ Var('b') → tab_idx:38, type:integer, lev:0

└─ Compound(0 stmts)

Tabel 5.2 Kasus Uji 2

Input							
<pre> program ValidTest; var x, y: integer; b: boolean; arr: array [1..10] of integer; rec: record val: integer; flag: boolean end; function add(val1: integer; val2: integer): integer; var temp: integer; begin temp := val1 + val2; add := temp; end; begin x := 5; y := 10; b := (x < y) and true; arr[1] := add(x, y); rec.val := 100; rec.flag := false; if b then writeln('Valid') else x := 0; while x > 0 do begin x := x - 1; end; end; end. </pre>							
Output							
SYMBOL TABLE							
IDX	ID	OBJ	TYPE	REF	NRM	LEV	
ADR	CONST_VAL						
0	and	0	0	0	1	0	0
1	array	3	5	0	1	0	0
2	begin	0	0	0	1	0	0

3	case	0	0	0	1	0	0
4	const	0	0	0	1	0	0
5	div	0	0	0	1	0	0
6	downto	0	0	0	1	0	0
7	do	0	0	0	1	0	0
8	else	0	0	0	1	0	0
9	end	0	0	0	1	0	0
10	for	0	0	0	1	0	0
11	function	4	0	0	1	0	0
12	if	0	0	0	1	0	0
13	mod	0	0	0	1	0	0
14	not	0	0	0	1	0	0
15	of	0	0	0	1	0	0
16	or	0	0	0	1	0	0
17	procedure	5	0	0	1	0	0
18	program	0	0	0	1	0	0
19	record	3	6	0	1	0	0
20	repeat	0	0	0	1	0	0
21	integer	3	1	0	1	0	0
22	real	3	2	0	1	0	0
23	boolean	3	3	0	1	0	0
24	char	3	4	0	1	0	0
25	string	3	7	0	1	0	0
26	then	0	0	0	1	0	0
27	to	0	0	0	1	0	0
28	type	0	0	0	1	0	0
29	until	0	0	0	1	0	0
30	var	0	0	0	1	0	0
31	while	0	0	0	1	0	0
32	true	1	3	0	1	0	1
true							
33	false	1	3	0	1	0	0
false							
34	readln	5	0	0	1	0	0
35	writeln	5	0	0	1	0	0
36	ValidTest	5	0	0	1	0	0
37	x	2	1	0	1	0	0
38	y	2	1	0	1	0	1
39	b	2	3	0	1	0	2
40	arr	2	5	1	1	0	3
41	val	2	1	0	1	1	0
42	flag	2	3	0	1	1	1
43	rec	2	6	1	1	0	4
44	add	4	1	2	1	0	0
45	val1	2	1	0	1	1	0
46	val2	2	1	0	1	1	1
47	temp	2	1	0	1	1	0

BLOCK TABLE

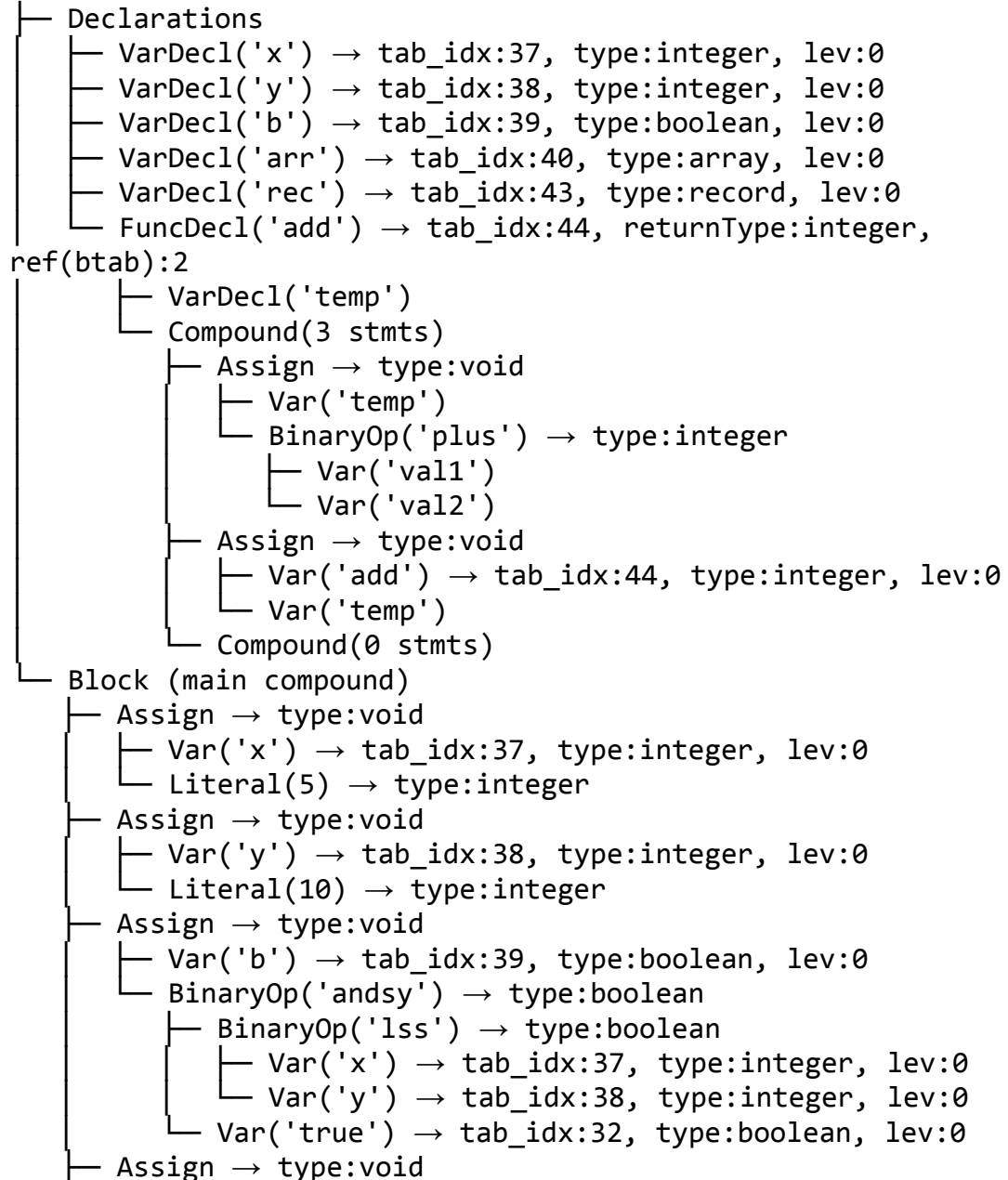
IDX	LAST	LPAR	PSZE	VSZE
0	44	0	0	5
1	42	0	0	0
2	47	46	2	1

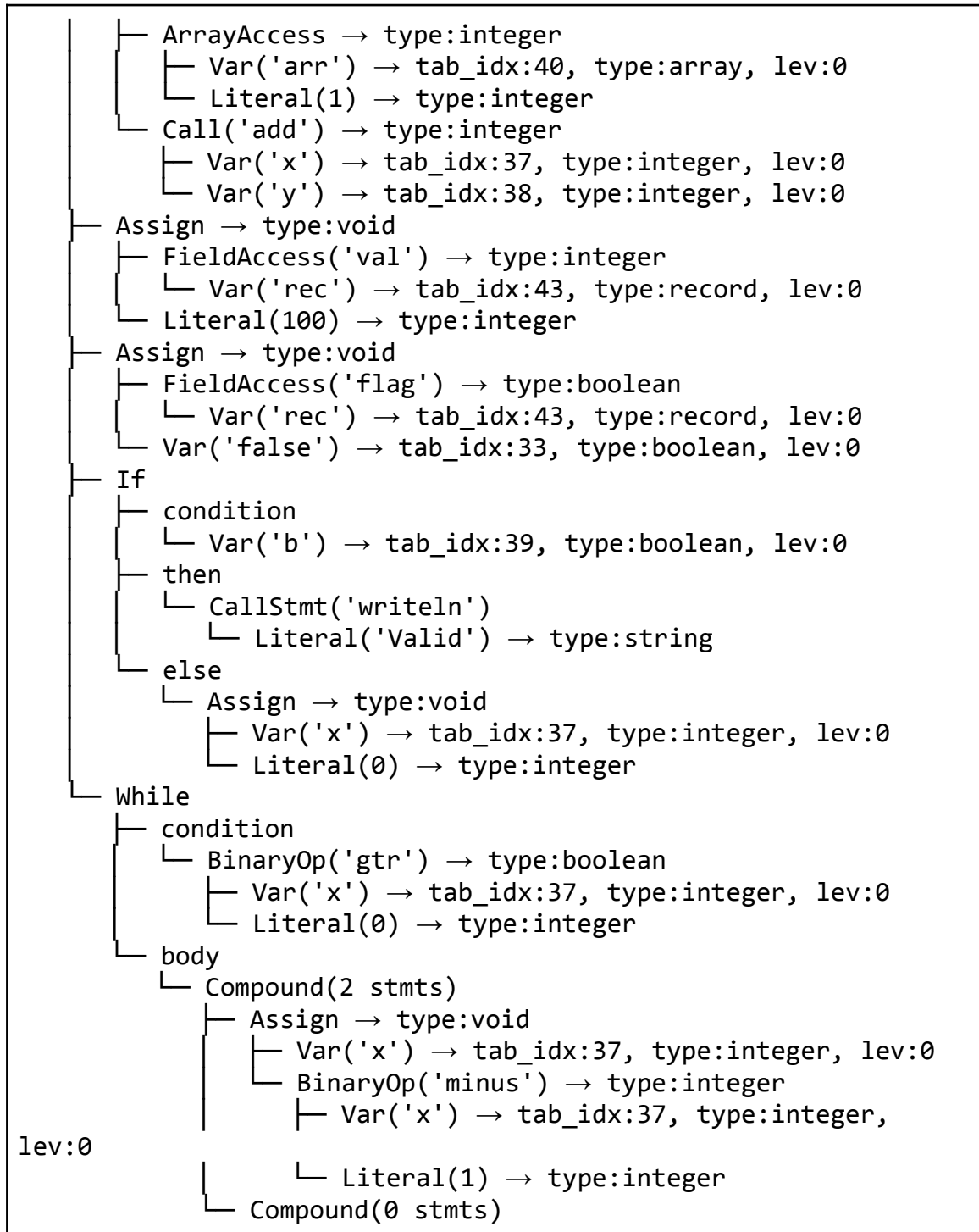
ARRAY TABLE

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
idx	xtyp	etyp	eref	low	high	elsz	size
0	1	1	0	1	10	1	10
1	1	1	0	1	10	1	10

=== DECORATED ABSTRACT SYNTAX TREE ===

ProgramNode(name: 'ValidTest')





❖ Kasus uji 3

Tabel 5.3 Kasus Uji 3

Input
<pre> program ScopeError; var </pre>

<pre> a: integer; a: real; { eror 1: Redeclaration of 'a' } begin a := 10; b := 20 { eror 2: Undeclared identifier 'b' } end. </pre>
Output
<pre> Semantic Error: Redeclaration of variable 'a' in current scope Semantic Error: Undeclared identifier 'b' </pre>

❖ Kasus uji 4

Tabel 5.4 Kasus Uji 4

Input
<pre> program OutOfScopeCheck; var globalVar: integer; procedure testProc; var localVar: integer; begin localVar := 1; globalVar := 2 end; begin globalVar := 10; localVar := 5 { error : out of scope } end. </pre>
Output
<pre> Semantic Error: Undeclared identifier 'localVar' </pre>

❖ Kasus uji 5

Tabel 5.5 Kasus Uji 5

Input
<pre> program ParamError; </pre>

```

var
    res: integer;

function multiply(val1, val2: integer): integer;
begin
    multiply := val1 * val2
end;

begin
    res := multiply(5, true); { eror 1: Tipe param 2 salah
(integer vs bool) }
    res := multiply(5)        { eror 2: Jumlah parameter kurang
}
end.

```

Output

```

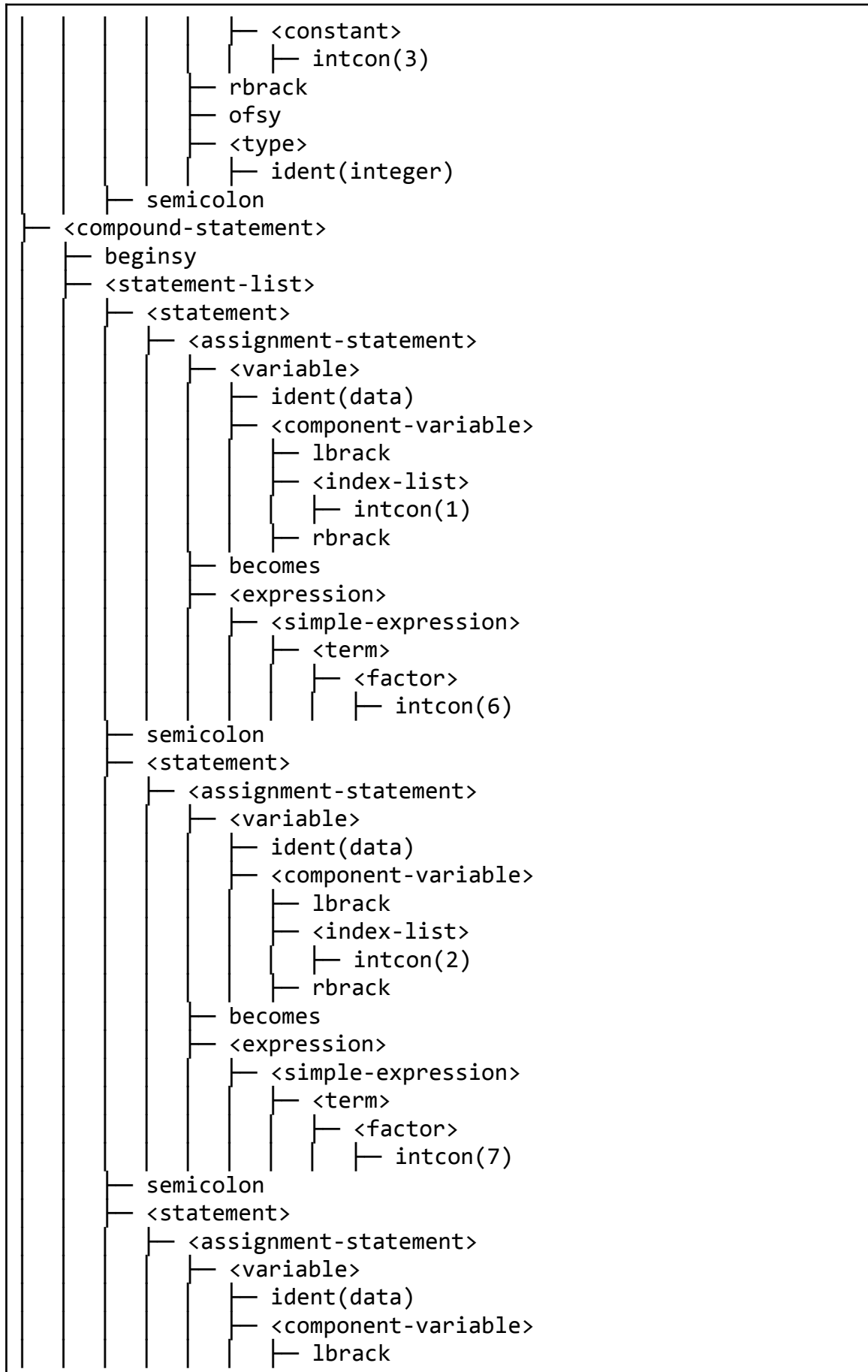
Semantic Error: Type mismatch for parameter 2 in call to 'multiply'. Expected integer, got boolean
Semantic Error: Incorrect number of arguments for 'multiply'. Expected 2, got 1

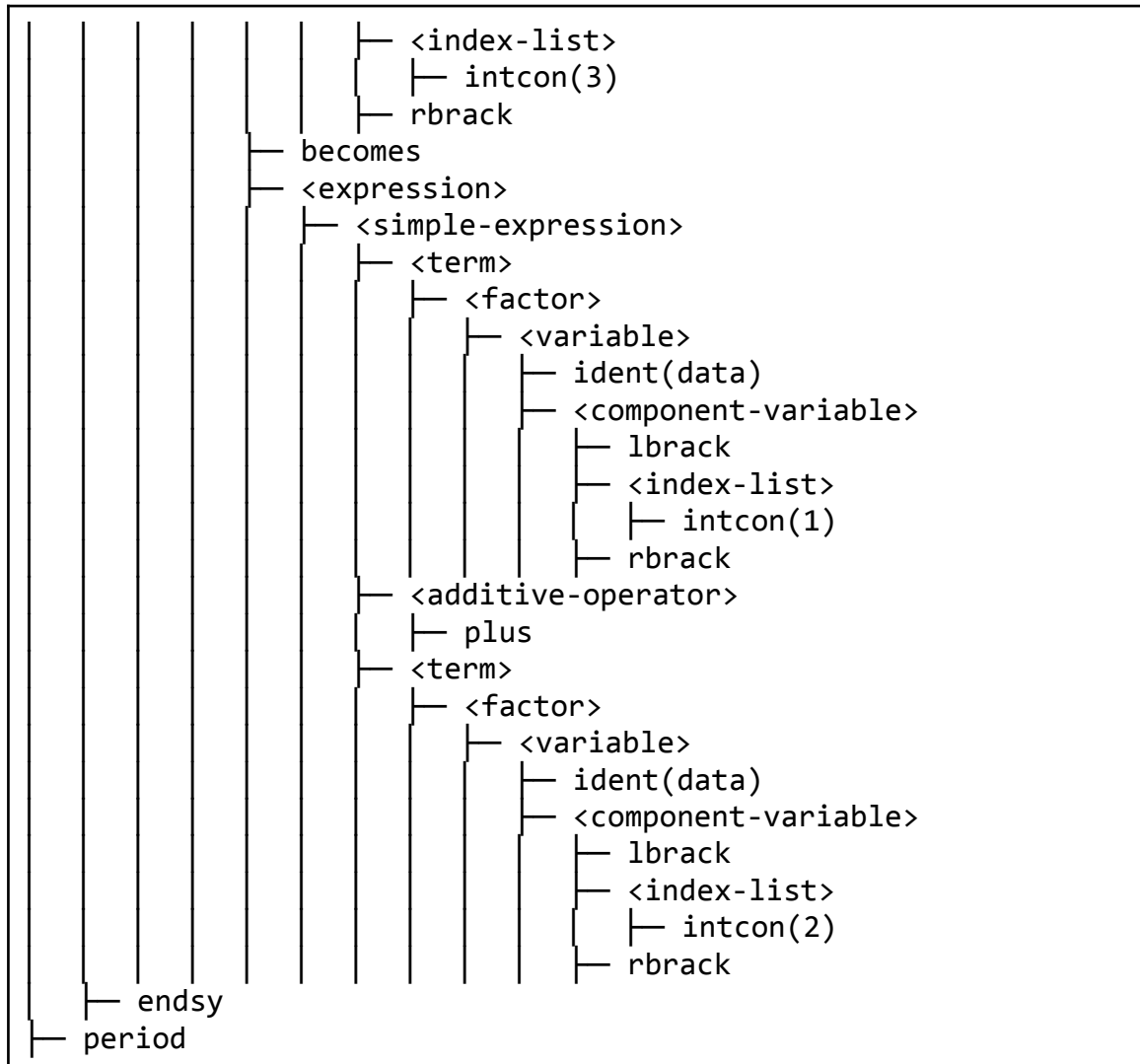
```

❖ Kasus uji 6

Tabel 5.6 Kasus Uji 6

Input
<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident(TestRangeArray) │ └── semicolon ├─ <declaration-part> │ ├── <var-declaration> │ │ ├── varsy │ │ ├── <identifier-list> │ │ │ └── ident(data) │ │ ├── colon │ │ ├── <type> │ │ │ ├── <array-type> │ │ │ │ ├── arraysy │ │ │ │ ├── lbrack │ │ │ │ └── <range> │ │ │ │ ├── <constant> │ │ │ │ │ └── intcon(1) │ │ │ │ └── period │ │ └── period </pre>





Output

SYMBOL TABLE

IDX	ID	OBJ	TYPE	REF	NRM	LEV
ADR	CONST_VAL					
0	and	0	0	0	1	0 0
1	array	3	5	0	1	0 0
2	begin	0	0	0	1	0 0
3	case	0	0	0	1	0 0
4	const	0	0	0	1	0 0
5	div	0	0	0	1	0 0
6	downto	0	0	0	1	0 0
7	do	0	0	0	1	0 0
8	else	0	0	0	1	0 0
9	end	0	0	0	1	0 0
10	for	0	0	0	1	0 0
11	function	4	0	0	1	0 0
12	if	0	0	0	1	0 0

13	mod	0	0	0	1	0	0
14	not	0	0	0	1	0	0
15	of	0	0	0	1	0	0
16	or	0	0	0	1	0	0
17	procedure	5	0	0	1	0	0
18	program	0	0	0	1	0	0
19	record	3	6	0	1	0	0
20	repeat	0	0	0	1	0	0
21	integer	3	1	0	1	0	0
22	real	3	2	0	1	0	0
23	boolean	3	3	0	1	0	0
24	char	3	4	0	1	0	0
25	string	3	7	0	1	0	0
26	then	0	0	0	1	0	0
27	to	0	0	0	1	0	0
28	type	0	0	0	1	0	0
29	until	0	0	0	1	0	0
30	var	0	0	0	1	0	0
31	while	0	0	0	1	0	0
32	true	1	3	0	1	0	1
true							
33	false	1	3	0	1	0	0
false							
34	readln	5	0	0	1	0	0
35	writeln	5	0	0	1	0	0
36	TestRangeArray	5	0	0	1	0	0
37	data	2	5	1	1	0	0

BLOCK TABLE

IDX	LAST	LPAR	PSIZE	VSZ
0	37	0	0	1

ARRAY TABLE

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
idx	xtyp	etyp	eref	low	high	elsz	size
0	1	1	0	1	3	1	3
1	1	1	0	1	3	1	3

=== DECORATED ABSTRACT SYNTAX TREE ===

ProgramNode(name: 'TestRangeArray')

├─ Declarations

├─ VarDecl('data') → tab_idx:37, type:array, lev:0

└─ Block (main compound)

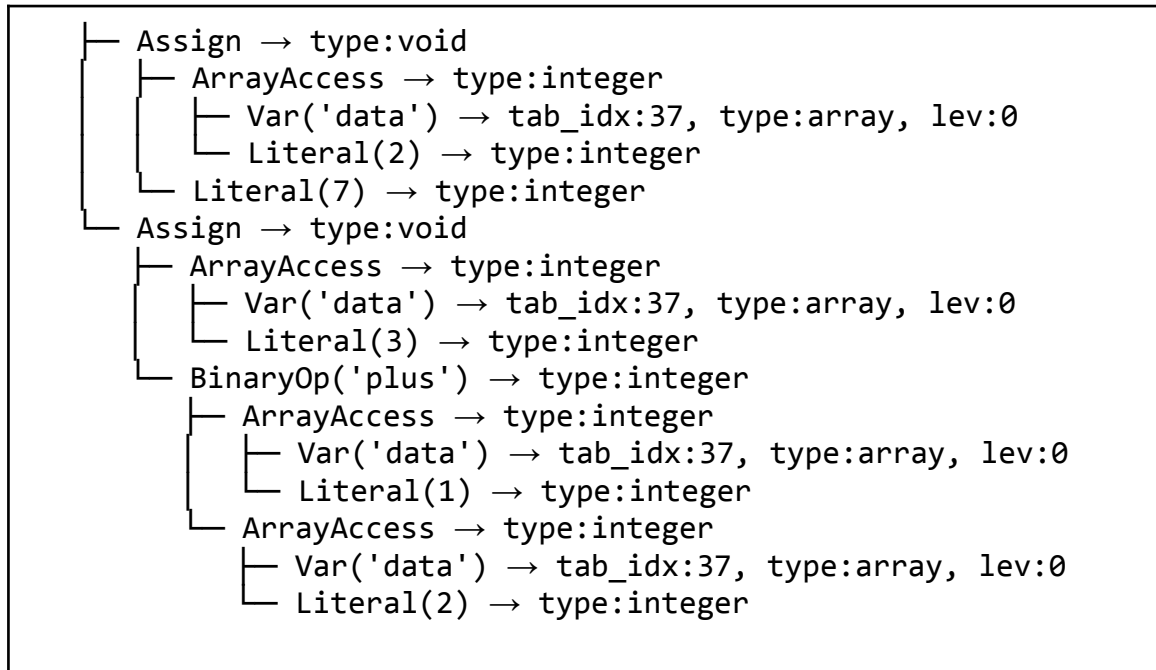
├─ Assign → type:void

├─ ArrayAccess → type:integer

├─ Var('data') → tab_idx:37, type:array, lev:0

├─ Literal(1) → type:integer

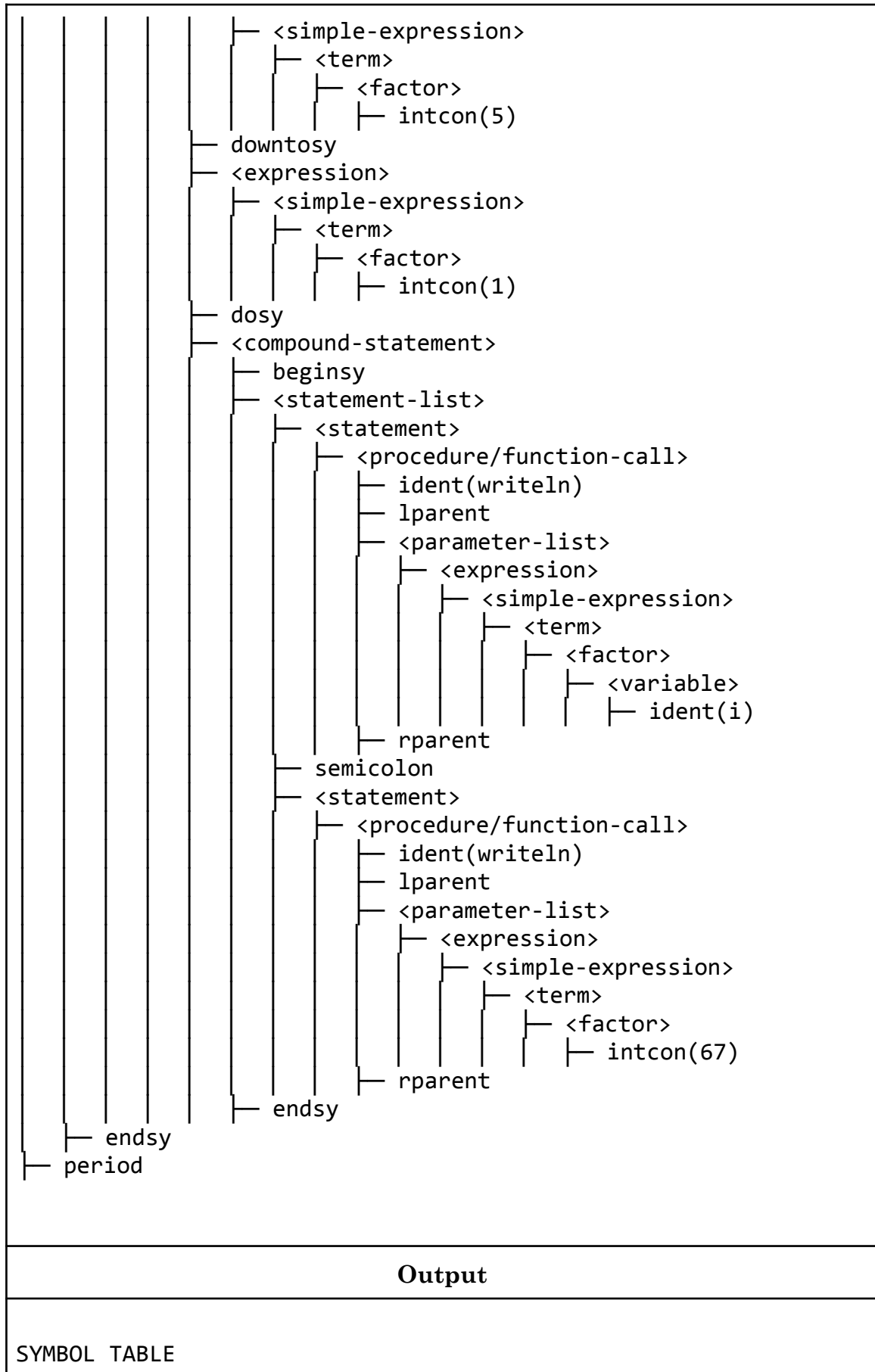
└─ Literal(6) → type:integer



❖ Kasus uji 7

Tabel 5.7 Kasus Uji 7

Input	
<pre> <program> ├─ <program-header> │ ├── programsy │ ├── ident(TestDownTo) │ └── semicolon ├─ <declaration-part> │ ├── <var-declaration> │ │ ├── varsy │ │ ├── <identifier-list> │ │ │ └── ident(i) │ │ ├── colon │ │ ├── <type> │ │ │ └── ident(integer) │ │ └── semicolon │ └─ <compound-statement> │ ├── beginsy │ └─ <statement-list> │ ├── <statement> │ │ └─ <for-statement> │ │ ├── forsy │ │ ├── ident(i) │ │ ├── becomes │ │ └─ <expression> </pre>	



IDX	ID	OBJ	TYPE	REF	NRM	LEV	
ADR	CONST_VAL						
0	and	0	0	0	1	0	0
1	array	3	5	0	1	0	0
2	begin	0	0	0	1	0	0
3	case	0	0	0	1	0	0
4	const	0	0	0	1	0	0
5	div	0	0	0	1	0	0
6	downto	0	0	0	1	0	0
7	do	0	0	0	1	0	0
8	else	0	0	0	1	0	0
9	end	0	0	0	1	0	0
10	for	0	0	0	1	0	0
11	function	4	0	0	1	0	0
12	if	0	0	0	1	0	0
13	mod	0	0	0	1	0	0
14	not	0	0	0	1	0	0
15	of	0	0	0	1	0	0
16	or	0	0	0	1	0	0
17	procedure	5	0	0	1	0	0
18	program	0	0	0	1	0	0
19	record	3	6	0	1	0	0
20	repeat	0	0	0	1	0	0
21	integer	3	1	0	1	0	0
22	real	3	2	0	1	0	0
23	boolean	3	3	0	1	0	0
24	char	3	4	0	1	0	0
25	string	3	7	0	1	0	0
26	then	0	0	0	1	0	0
27	to	0	0	0	1	0	0
28	type	0	0	0	1	0	0
29	until	0	0	0	1	0	0
30	var	0	0	0	1	0	0
31	while	0	0	0	1	0	0
32	true	1	3	0	1	0	1
true							
33	false	1	3	0	1	0	0
false							
34	readln	5	0	0	1	0	0
35	writeln	5	0	0	1	0	0
36	TestDownTo	5	0	0	1	0	0
37	i	2	1	0	1	0	0
BLOCK TABLE							
IDX	LAST	LPAR	PSZE	VSZE			
0	37	0	0	1			
ARRAY TABLE							

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
idx	xtyp	etyp	eref	low	high	elsz	size

=== DECORATED ABSTRACT SYNTAX TREE ===

```

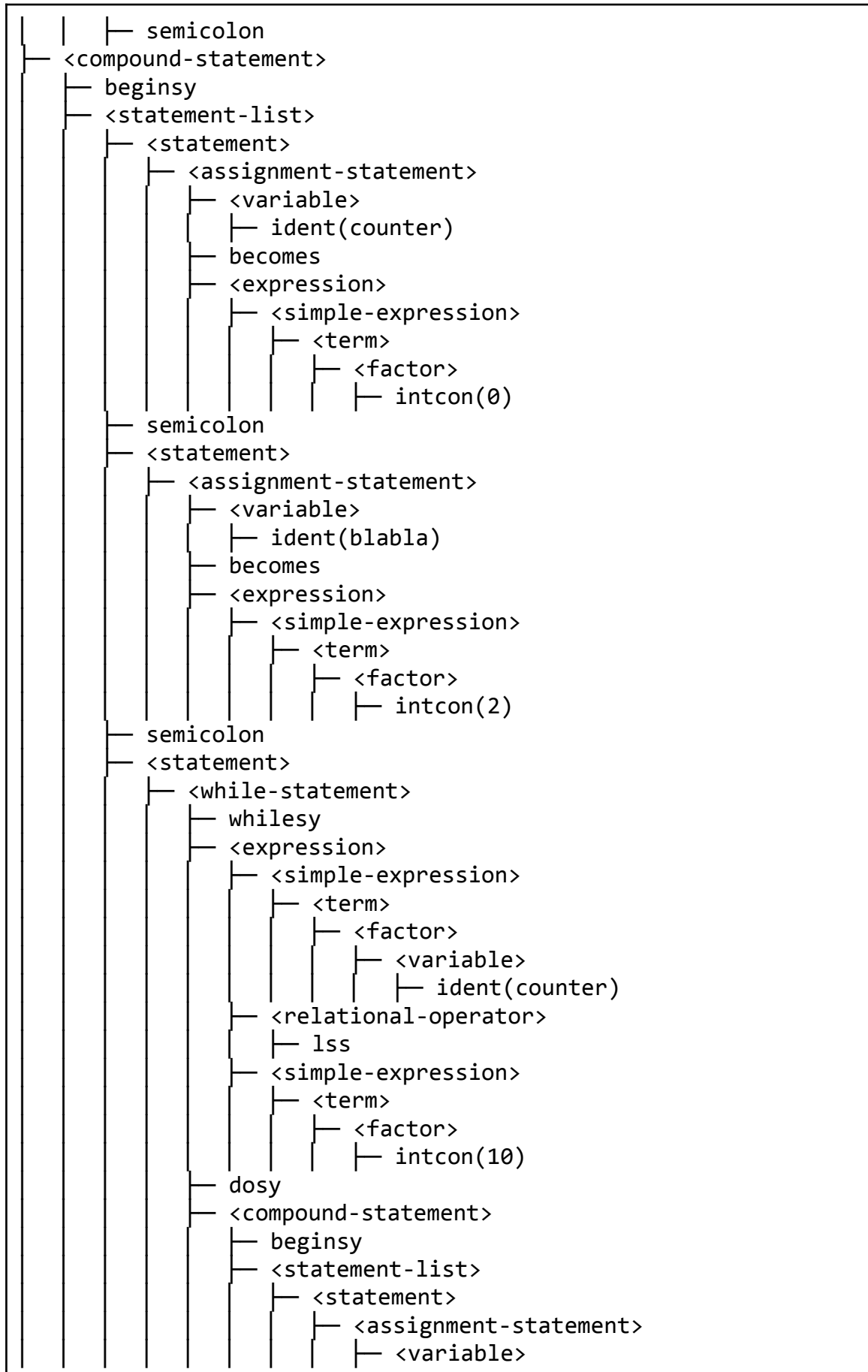
ProgramNode(name: 'TestDownTo')
├── Declarations
│   └── VarDecl('i') → tab_idx:37, type:integer, lev:0
├── Block (main compound)
│   └── For('i' downto)
│       ├── start
│       │   └── Literal(5) → type:integer
│       ├── end
│       │   └── Literal(1) → type:integer
│       └── body
│           └── Compound(2 stmts)
│               ├── CallStmt('writeln')
│               │   └── Var('i') → tab_idx:37, type:integer, lev:0
│               └── CallStmt('writeln')
│                   └── Literal(67) → type:integer

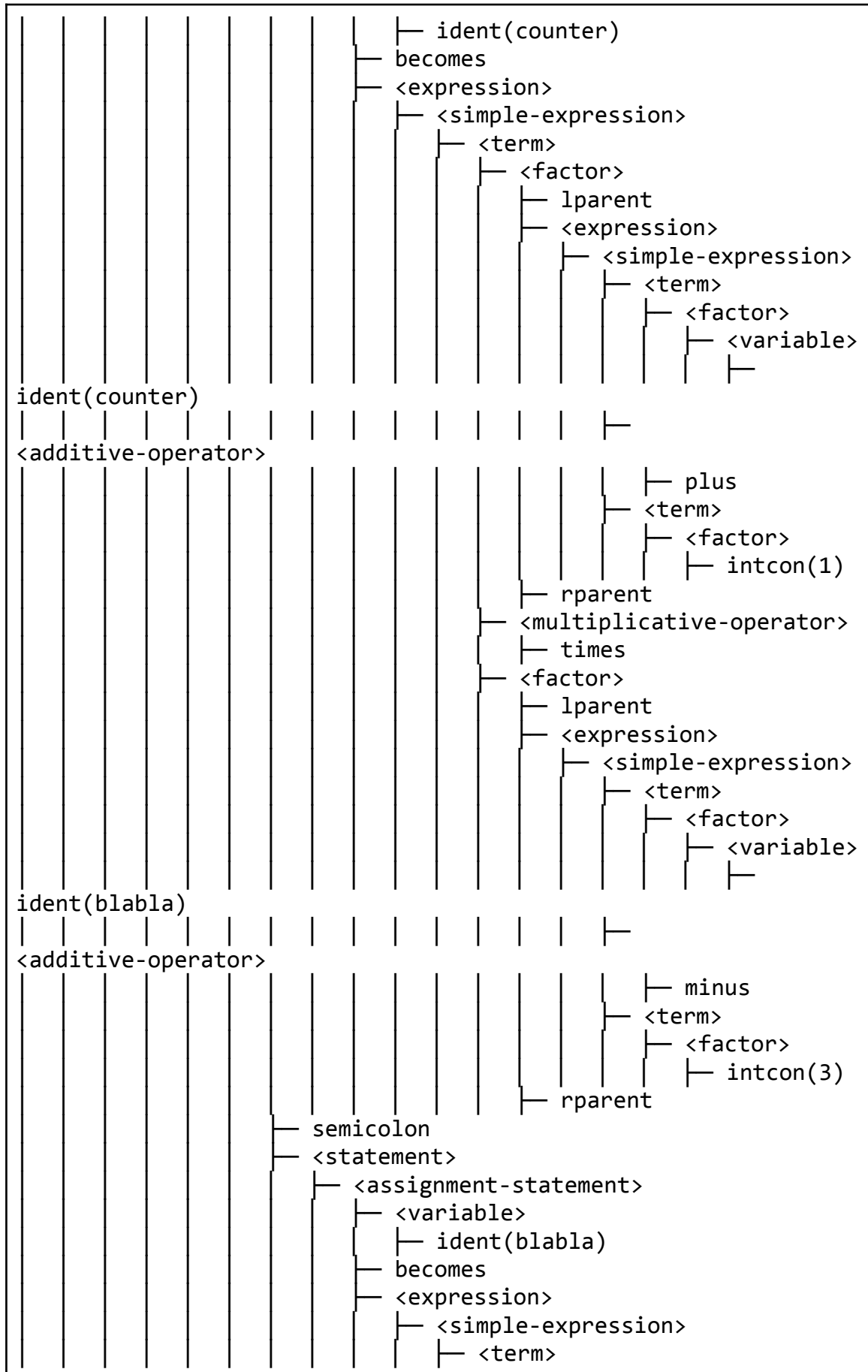
```

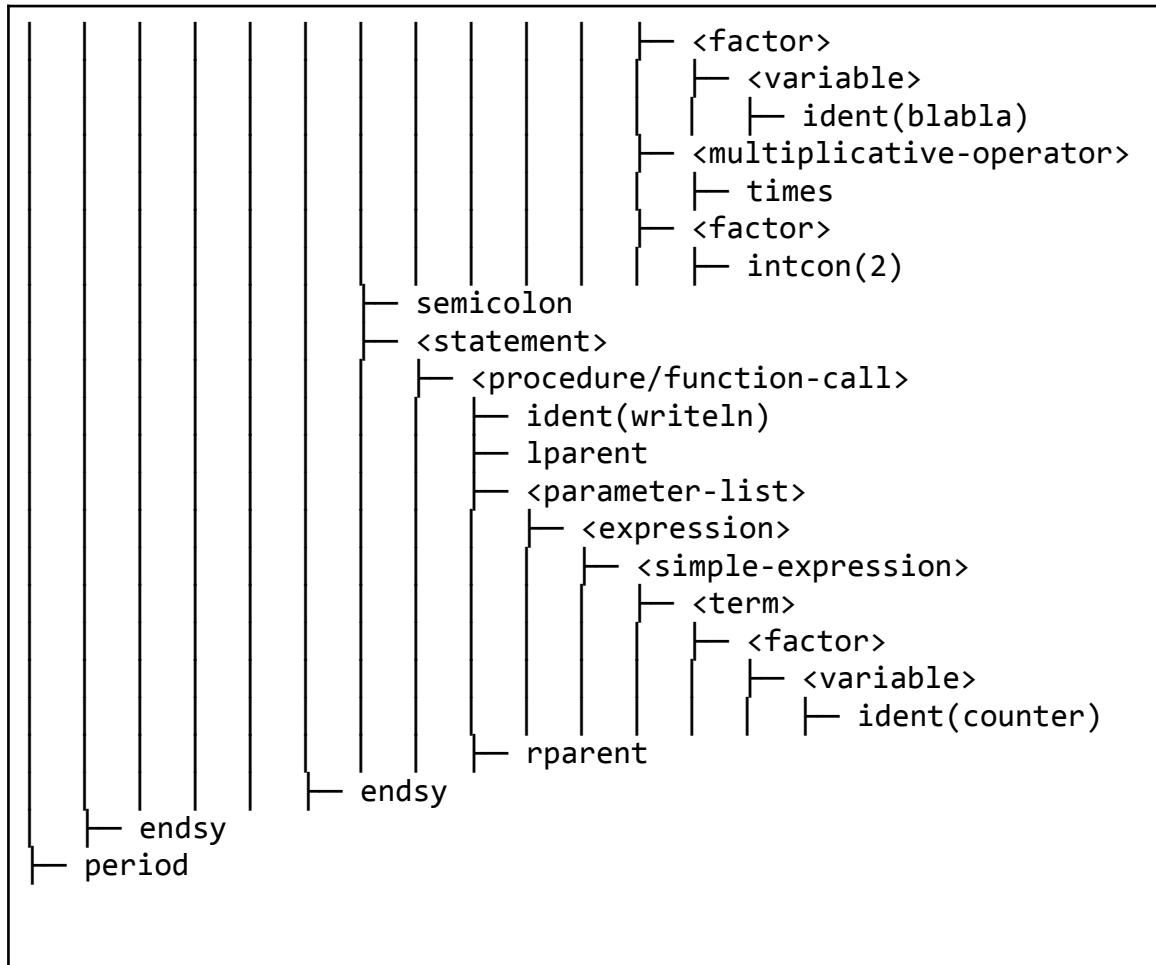
❖ Kasus uji 8

Tabel 5.8 Kasus Uji 8

Input	
<pre> <program> ├── <program-header> │ ├── programsy │ ├── ident(TestIntegerOps) │ └── semicolon ├── <declaration-part> │ ├── <var-declaration> │ │ ├── varsy │ │ ├── <identifier-list> │ │ │ └── ident(counter) │ │ ├── colon │ │ ├── <type> │ │ │ └── ident(integer) │ │ ├── semicolon │ │ ├── <identifier-list> │ │ │ └── ident(blalba) │ │ ├── colon │ │ ├── <type> │ │ │ └── ident(integer) </pre>	







Output

SYMBOL TABLE

IDX	ID	OBJ	TYPE	REF	NRM	LEV
ADR	CONST_VAL					
0	and	0	0	0	1	0
1	array	3	5	0	1	0
2	begin	0	0	0	1	0
3	case	0	0	0	1	0
4	const	0	0	0	1	0
5	div	0	0	0	1	0
6	downto	0	0	0	1	0
7	do	0	0	0	1	0
8	else	0	0	0	1	0
9	end	0	0	0	1	0
10	for	0	0	0	1	0
11	function	4	0	0	1	0
12	if	0	0	0	1	0
13	mod	0	0	0	1	0
14	not	0	0	0	1	0
15	of	0	0	0	1	0

16	or	0	0	0	1	0	0
17	procedure	5	0	0	1	0	0
18	program	0	0	0	1	0	0
19	record	3	6	0	1	0	0
20	repeat	0	0	0	1	0	0
21	integer	3	1	0	1	0	0
22	real	3	2	0	1	0	0
23	boolean	3	3	0	1	0	0
24	char	3	4	0	1	0	0
25	string	3	7	0	1	0	0
26	then	0	0	0	1	0	0
27	to	0	0	0	1	0	0
28	type	0	0	0	1	0	0
29	until	0	0	0	1	0	0
30	var	0	0	0	1	0	0
31	while	0	0	0	1	0	0
32	true	1	3	0	1	0	1
33	false	1	3	0	1	0	0
34	readln	5	0	0	1	0	0
35	writeln	5	0	0	1	0	0
36	TestIntegerOps	5	0	0	1	0	0
37	counter	2	1	0	1	0	0
38	blabla	2	1	0	1	0	1

BLOCK TABLE

IDX	LAST	LPAR	PSZE	VSZE
0	38	0	0	2

ARRAY TABLE

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
idx	xtyp	etyp	eref	low	high	elsz	size

=== DECORATED ABSTRACT SYNTAX TREE ===

ProgramNode(name: 'TestIntegerOps')

Declarations

VarDecl('counter') → tab_idx:37, type:integer, lev:0

VarDecl('blabla') → tab_idx:38, type:integer, lev:0

Block (main compound)

Assign → type:void

Var('counter') → tab_idx:37, type:integer, lev:0

Literal(0) → type:integer

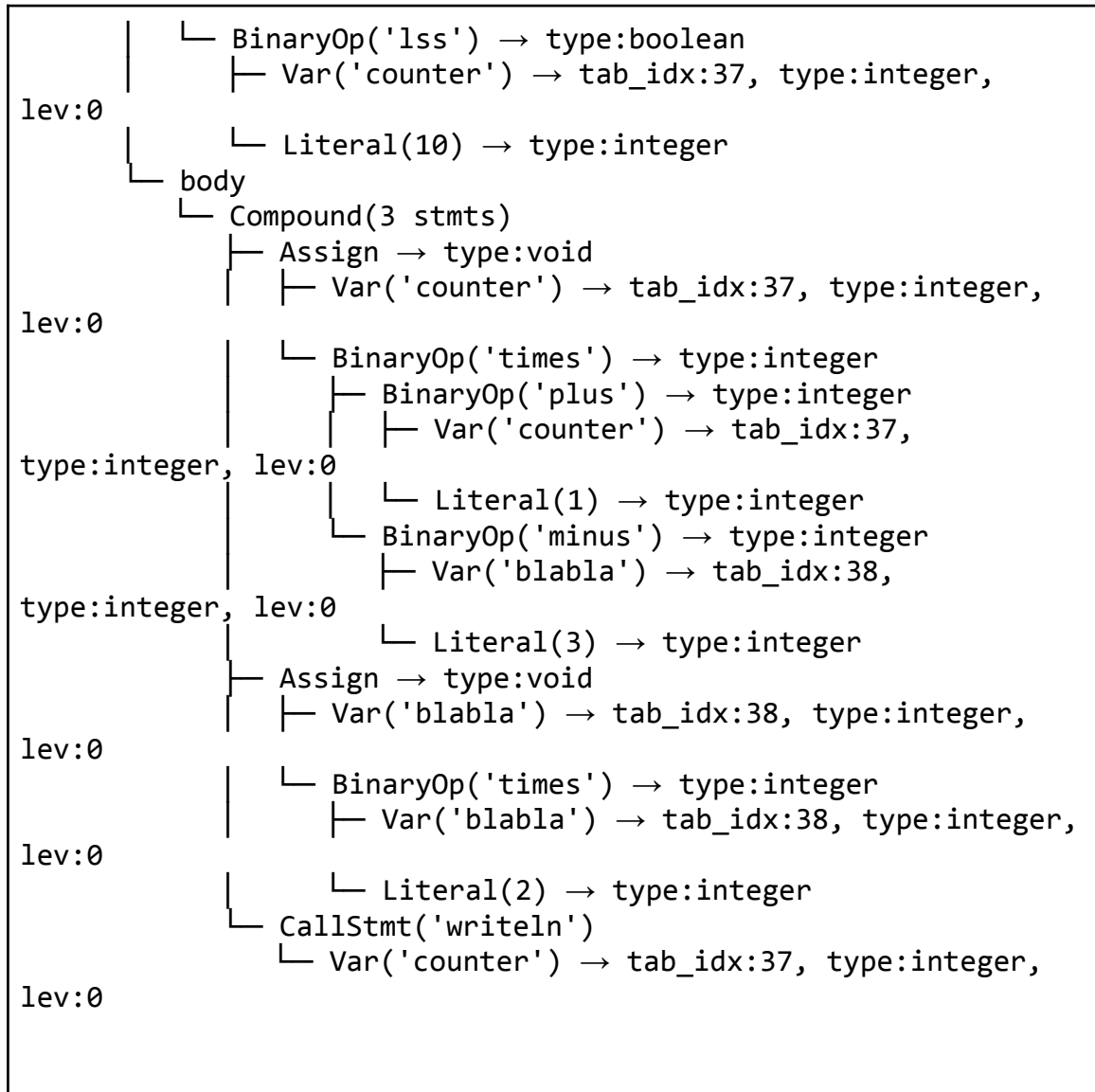
Assign → type:void

Var('blabla') → tab_idx:38, type:integer, lev:0

Literal(2) → type:integer

While

condition



BAB 6

KESIMPULAN DAN SARAN

6.1. Kesimpulan

Pada milestone ini telah berhasil diimplementasikan tahap ketiga dari pipeline compiler untuk bahasa Arion, yaitu Semantic Analysis. Proses ini terdiri dari dua tahap utama: (1) Konversi Parse Tree menjadi Abstract Syntax Tree (AST) menggunakan Syntax-Directed Translation dengan 43 produksi grammar yang masing-masing dilengkapi *semantic action*, (2) Dekorasi AST menjadi Decorated AST melalui penelusuran DFS menggunakan fungsi visit yang mengelola scope, melakukan registrasi dan lookup identifier, serta melakukan inferensi dan validasi tipe data.

Symbol Table yang digunakan terdiri dari tiga tabel utama, yaitu `tab`, `btab`, dan `atab`, yang masing-masing menyimpan informasi identifier, blok scope, dan tipe array. Manajemen scope dilakukan menggunakan struktur *stack* dengan mekanisme `enterScope` dan `exitScope`, sehingga pencarian identifier dapat dilakukan dari scope terdalam hingga terluar. Selain identifier yang dideklarasikan oleh programmer, sistem juga menyediakan 11 predefined identifier yang mencakup tipe data primitif, konstanta bawaan, serta prosedur standar `readln` dan `writeln`.

Pengecekan semantik yang dilakukan mencakup pengecekan redeklaasi identifier, validasi penggunaan identifier yang belum dideklarasikan, inferensi tipe pada ekspresi, serta validasi kompatibilitas tipe pada operasi aritmatika, logika, relasional, dan assignment. Hasil akhir dari proses ini adalah Decorated AST beserta Symbol Table yang siap digunakan oleh tahap berikutnya dalam pipeline compiler.

6.2. Saran

Untuk pengembangan selanjutnya, terdapat beberapa hal yang dapat ditingkatkan. Pertama, pengecekan pada `for statement` saat ini belum memvalidasi bahwa variabel loop harus bertipe Integer atau Char. Validasi ini perlu ditambahkan untuk memastikan kebenaran semantik yang lebih ketat. Kedua, parameter passing saat ini belum dilakukan pengecekan jumlah dan tipe argumen terhadap deklarasi formal parameter prosedur atau fungsi yang dipanggil. Validasi ini penting untuk memastikan pemanggilan fungsi dan prosedur yang benar. Ketiga, sistem type compatibility saat ini belum menangani kompatibilitas antara tipe yang didefinisikan pengguna seperti `enumerated` dan `record` secara penuh, sehingga perlu dikembangkan lebih lanjut sesuai kebutuhan bahasa Arion yang lebih lengkap.

LAMPIRAN

Repository GitHub: <https://github.com/SyaqinaOctavia/MMD-Tubes-IF2224-2026.git>

NIM	Kontribusi Tugas	Persentase
13524040	Testing program, Testcase input-output, Semantic Analyzer, metode dalam ASTer	25%
13524042	Method parseTree file reader, metode dalam ASTer, Penulisan laporan	25%
13524048	Header AST, metode dalam ASTer, bugfixing, print Decorated AST	25%
13524088	Implementasi Symbol Table, Penulisan laporan	25%

REFERENSI

- John E., Rajeev M., dan Jeffrey D. *Introduction to Automata Theory, Languages, and Computation 3rd Edition*.
<https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613-Introduction-to-Automata-Theory,-Languages,-and-Computation-Edit-ion-3.pdf>
- Ruslan Spivak. *Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees*. Diakses 22 Mei 2026.
<https://ruslanspivak.com/lrbasi-part7>
- Ray Toal. *Intermediate Representations*. Diakses 22 Mei 2026.
<https://cs.lmu.edu/~ray/notes/ir/>
- Beck Hasti. *Syntax-Directed Translation*. Diakses 23 Mei 2026.
<https://pages.cs.wisc.edu/~hasti/cs536/readings/SDT.html>
- Geeks for Geeks. *S - Attributed and L - Attributed SDTs in Syntax Directed Translation*. Diakses 21 Mei 2026.
<https://www.geeksforgeeks.org/compiler-design/s-attributed-and-l-attributed-sdts-in-syntax-directed-translation/>